

智慧物流实验手册

文档版本 01
发布日期 2024-07-05



版权所有 © 华为云计算技术有限公司 2023。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HUAWEI和其他华为商标均为华为技术有限公司的商标。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为云计算技术有限公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为云计算技术有限公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

智慧物流

第一章 实验准备

1.1. 实验描述

本实验的目标是使用 YOLOv5 模型对四种物流产品进行分类识别，机器狗根据不同产品类型做出相应回应，智慧物流提高产业运输效能，在实际生产中，大大提高了运输效率。YOLOv5 是一个计算机视觉模式设计的对象检测任务，它是 YOLO 系列的升级版，YOLO 系列以其实时物体检测能力而著称。在实验过程中，通过海康工业相机收集大量物流产品数据集，对预训练权重进行模型训练和调优，转换模型格式，在昇腾开发板上进行推理，最终达到理想效果。

1.2. 实验目的

- 掌握智慧物流的实现流程，掌握 YOLOv5 训练全过程
- 掌握 labelImg 的使用，及数据集标注的使用方式
- 掌握昇腾开发板推理神经网络模型的方法
- 如何使用华为云 modelarts 进行模型训练

1.3. 实验建议

本实验主要是在 YOLOv5 上用 Python 编程，模型转换过程可以在自己的 PC 上进行，程序执行则需要在华为昇腾 Atlas200 开发板（Linux）上执行。实验中未注明的执行地方的，一律在开发板上执行。

前置基础课：

- (1) 数据结构与算法
- (2) 神经网络与深度学习

(3) 计算机视觉技术

(4) Python 数据分析

(5) 机器学习

(6) 数据集标注技术

1.4. 实验环境

本次实验使用的硬件包括：PC 机、昇腾 Atlas 200I DK 开发板、摄像头。实验开始前，请按照如下步骤将各硬件连接：

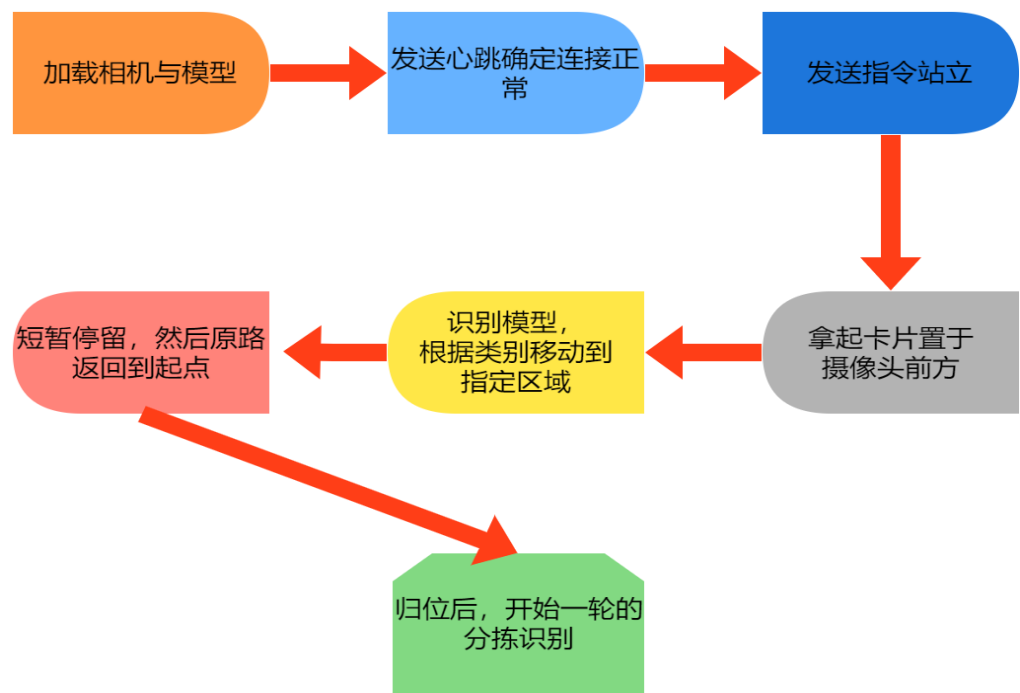
1. 使用数据线将开发板连接至 PC 机；
2. 将摄像头连接至开发板；
3. 给开发版供电，确保其能正常开机；

1.5. 实验道具讲解



四张卡片，表示的物流的 4 个种类 vegetables，drink，furniture，mechanical，在场景中通过摄像头来识别卡片的种类

第二章 实验过程描述



第三章 环境准备

3.1. 硬件环境准备

(1) SD 卡中系统烧录

向 SD 卡中烧录镜像的步骤如下。

- ①将 SD 卡插入读卡器，再将读卡器插到 PC 上。



②在浏览器中进入 [Ascend 官网](#)，选择“产品”->“开发者套件”



③点击“资源”，进入资源下载界面



④选择“制卡工具下载”，下载完成后进行安装。

寻你所需，玩转AI开发

快速入门

准备

请提前准备好下列物品，或查看 [硬件准备清单](#)，快速上手不卡顿

- MicroSD卡（推荐64GB）及读卡器（可选）
- 获取制卡工具，一键烧录镜像完成开发环境准备

制卡工具下载

校验文件下载

入门

提供入门 Atlas 200I DK A2 的全流程指导，帮助你30分钟内轻松完成从启动开发套件到运行首样例的过程

快速开始（课程）

快速开始（文档）

参考工具

提供Atlas 200I DK A2 进一步开发所需的参考工具及配套资源，可按需下载

模型适配工具

配套Conda环境包

使能开发者0代码完成数据标注、模型训练、模型部署到昇腾开发者套件

模型适配工具运行所必须的配套conda依赖环境，需要提前安装conda并导入conda环境包

软件包下载

校验文件下载

⑤打开镜像烧录工具 Ascend AI Devkit Imager，选择 Altas 200I DK A2

⑥打开“配置网络信息”，选择“Type-C”选项卡。可以看到 Type-C 接口默认 IP 为 192.168.0.2。选择默认 IP。

Error! Unknown document property name.

镜像网络信息配置

×

以下镜像默认IP信息，您可以根据PC/路由器IP信息重新填写网络信息

网口信息

ETH0

ETH1

Type-C

☐ 自动获取IP地址

☒ 使用下面的IP地址

IP地址

192 · 168 · 0 · 2

子网掩码

255 · 255 · 255 · 0

默认网关

· · ·

首选DNS服务器

8 · 8 · 8 · 8

备用DNS服务器

114 · 114 · 114 · 114

确定

取消

⑦选择要烧录的 SD 卡

昇腾开发者套件一键制卡工具

...

×

1 选择操作

在线制卡

本地制卡

备份SD卡

镜像版本

Desktop-Image_23.0.RC3_ubuntu22.04

Atlas 200I DK A2

配置网络信息

镜像包含OS、NPU驱动组件、CANN、MindX SDK、图形桌面、远程桌面、USB Audio...

OS版本: Ubuntu 22.04 LTS Arm64

固件与驱动版本: 23.0.RC3

CANN版本: 7.0.RC1

发布时间: 2023-11-15

缓存存在本地磁盘

镜像版本

Base-Image_23.0.RC1_ubuntu18.04

Atlas 200 DK

Atlas 200DK镜像,预装Python、CANN、MindX SDK、SSH、Samba、VNC、XRD...

OS版本: Ubuntu 18.04 LTS Arm64

固件与驱动版本: 1.0.13.alpha

CANN版本: 5.1.RC2.alpha007

发布时间: 2022-11-28

需要下载: 8.7 GB

2 选择SD卡

SD卡大小 62.5 GB

Mass Storage Device USB Device

Mass Storage Device USB Device-62.5 GB

挂载到: F:\, H:\, I\

Kingston DataTraveler 3.0 USB Device-31.0 GB

挂载到: G:\

3 烧录镜像

点击按钮烧录镜像

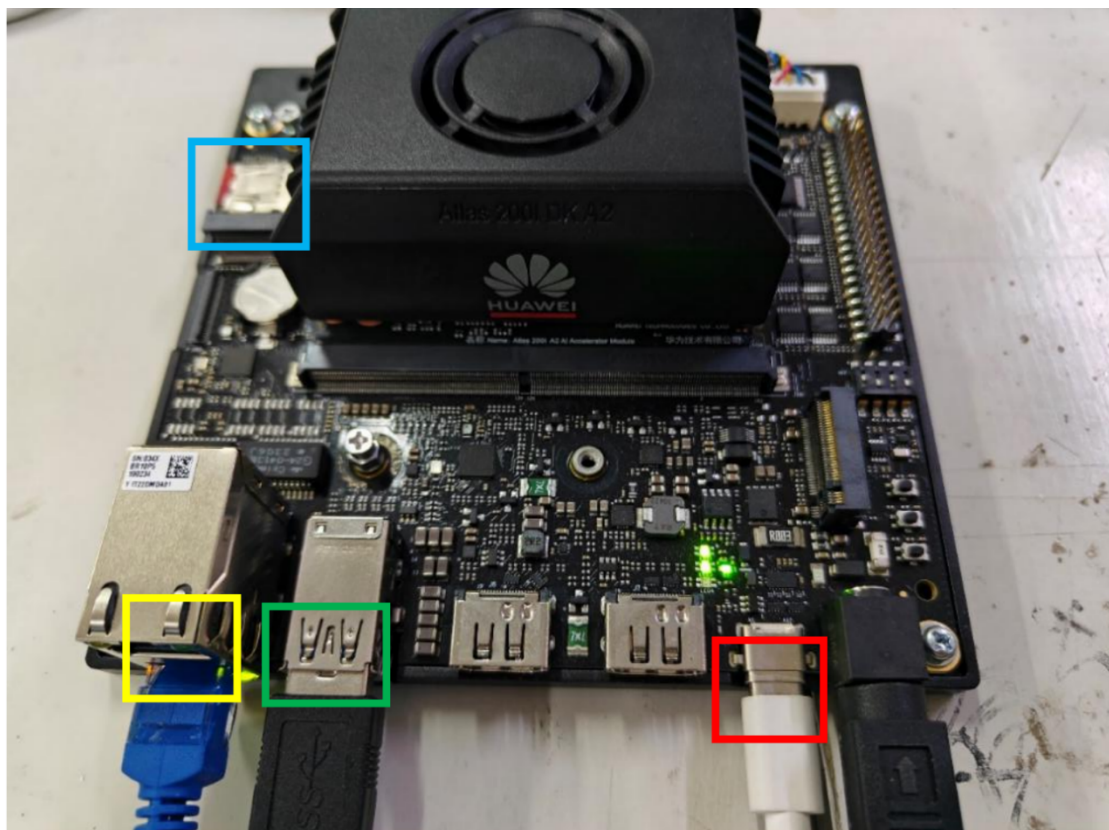
烧录镜像

⑧点击“烧录镜像”，等待镜像烧录完成。



(2) 开发板连接

将烧录好系统的 SD 卡插入到开发板中（下方蓝色框中），昇腾开发板和 PC 机通过 Type-C 线连接，将 Type-C 线和电源线连接到开发板（下方红色框中），并将海康工业相机和机器狗的 USB 线连接到开发板（下方绿色框中）。同时可以连接网线（下方黄色框中），让开发版连接互联网，方便环境依赖包的下载。连接方式如下图所示。



（3）PC 端远程连接开发板

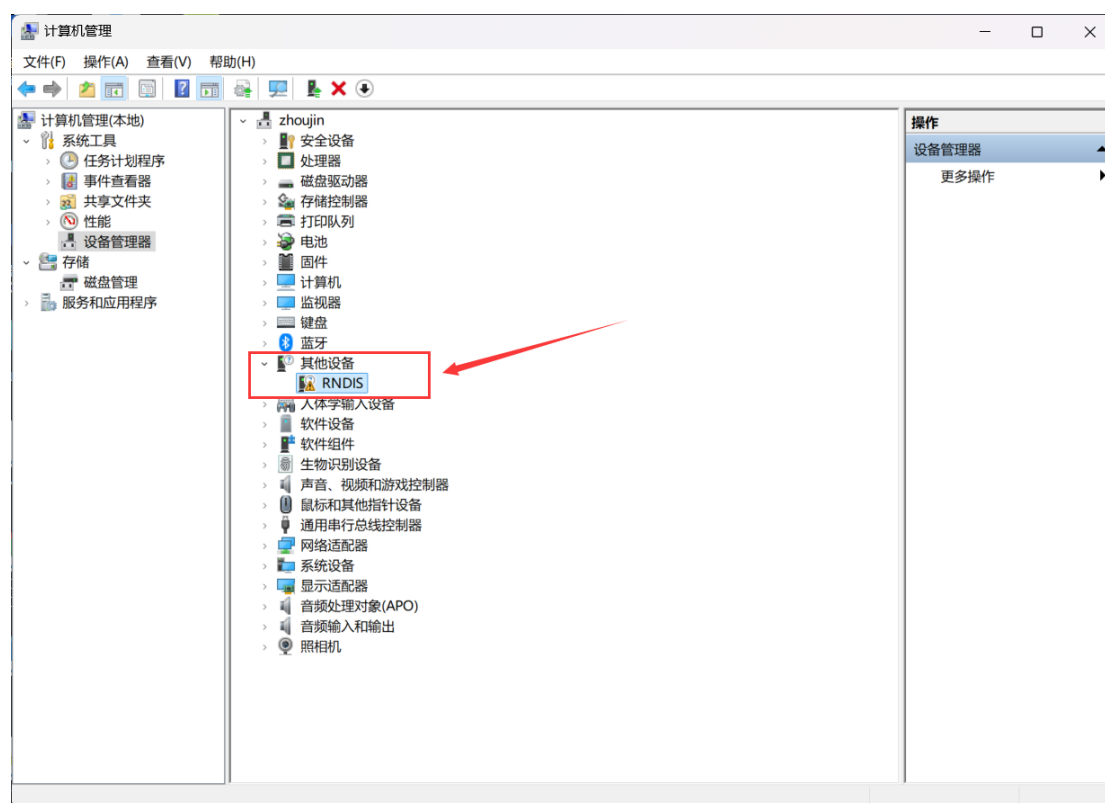
本节介绍当开发者套件通过 Type-C 接口和 PC 连接时，如何在 PC 将接口 IP 地址修改为和开发者套件接口 IP 地址同一个网段（如开发者套件 Type-C 接口为 192.168.0.2，PC 对应 USB 或 Type-C 接口为 192.168.0.101），并使用 SSH 工具远程登录开发者套件。

1) 安装 Windows 的 USB 网卡驱动

本步骤以 Windows 10 系统为例。

①在 PC 上右键单击“此电脑”，选择“更多”→“管理”，进入“计算机管理”界面。

②在“计算机管理”操作界面中选择“设备管理器→“其他设备”，如图所示，RNDIS 为未识别状态。



注意：如果未出现 RNDIS，请按以下方法排查：

- 确保开发者套件 Type-C 接口和 PC 已通过连接线正常连接。
- 更换具备数据传输能力的 Type-C 数据线（一般手机充电线拥有数据传输能力），继续查看是否出现 RNDIS。

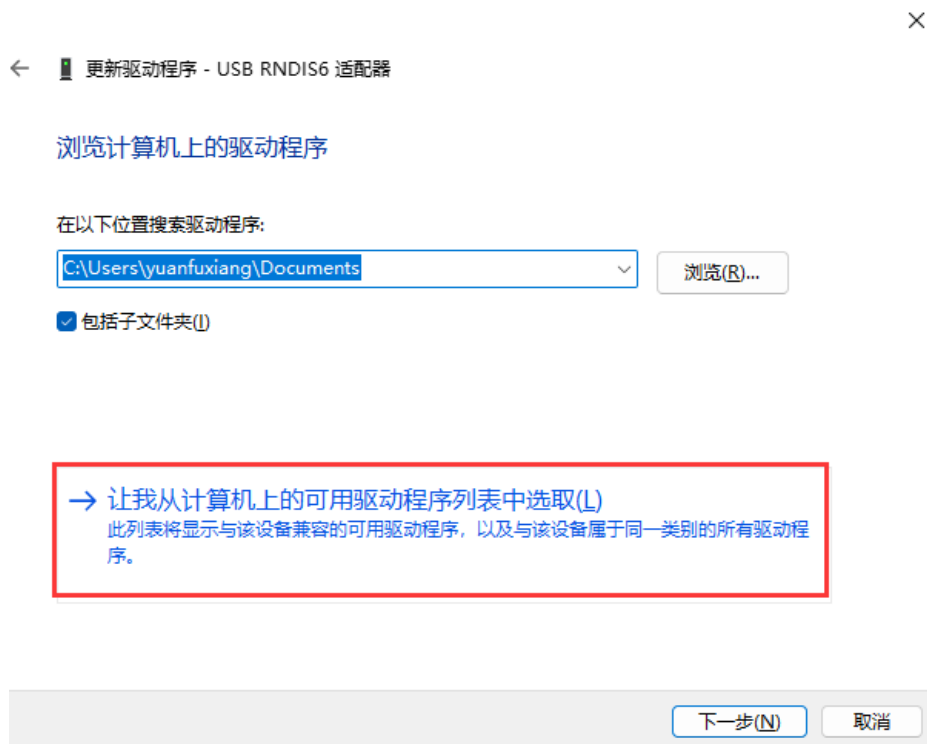
- c) 如果尝试现场所有 Type-C 数据线，仍无法出现 RNDIS，则考虑使用 RJ45 网线连接 PC 和开发者套件的以太网口的方式登录开发者套件。

③右键单击“RNDIS”，选择“更新驱动程序(P)”。

④在弹出的“更新驱动程序”→“RNDIS 窗口”中选择“浏览我的计算机以查找驱动程序软件(R)”。



⑤然后选择“让我从计算机上的可用驱动程序列表中选择(L)”。



⑥在“常见硬件类型”列表中选择“网络适配器”，单击“下一页(N)”。

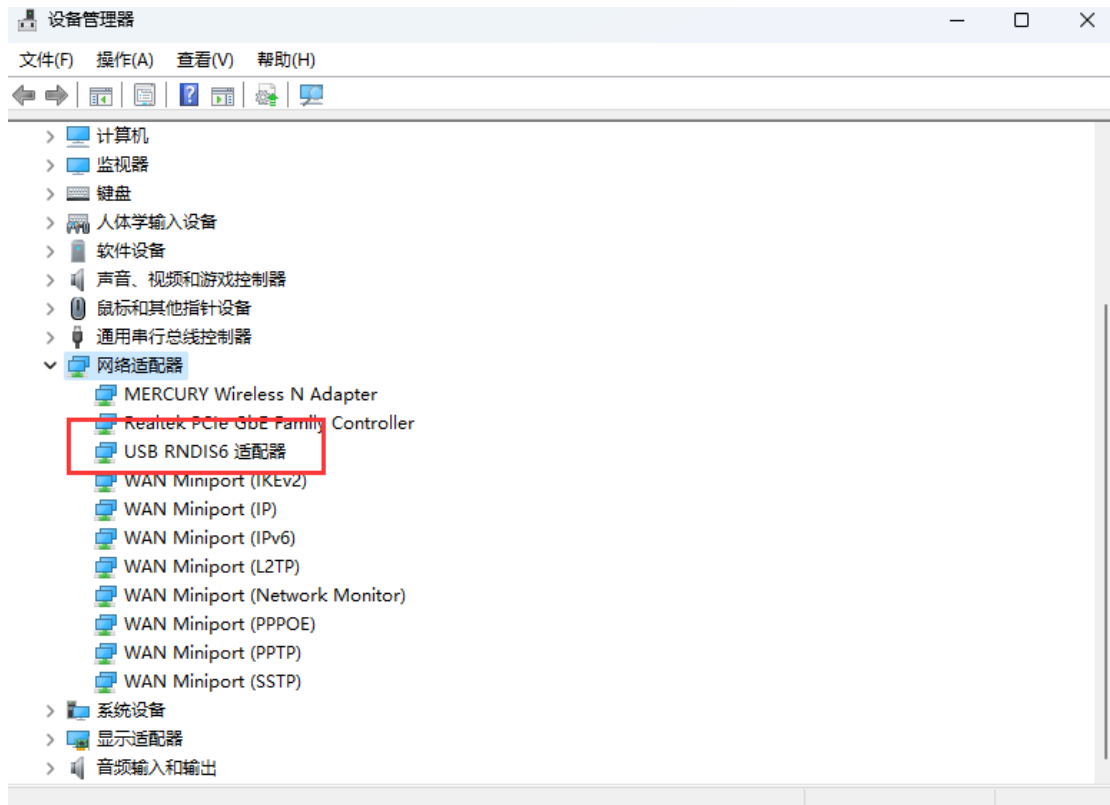


⑦在“选择要为此硬件安装的设备驱动程序”界面中选择“Microsoft”厂商的“USB RNDIS6 适配器”。



⑧单击“下一页”，在弹出的“更新驱动程序警告”窗口选择“是”。

⑨返回“设备管理器”→“网络适配器”，可看到已经正常显示了 USB RNDIS6 适配器的驱动。

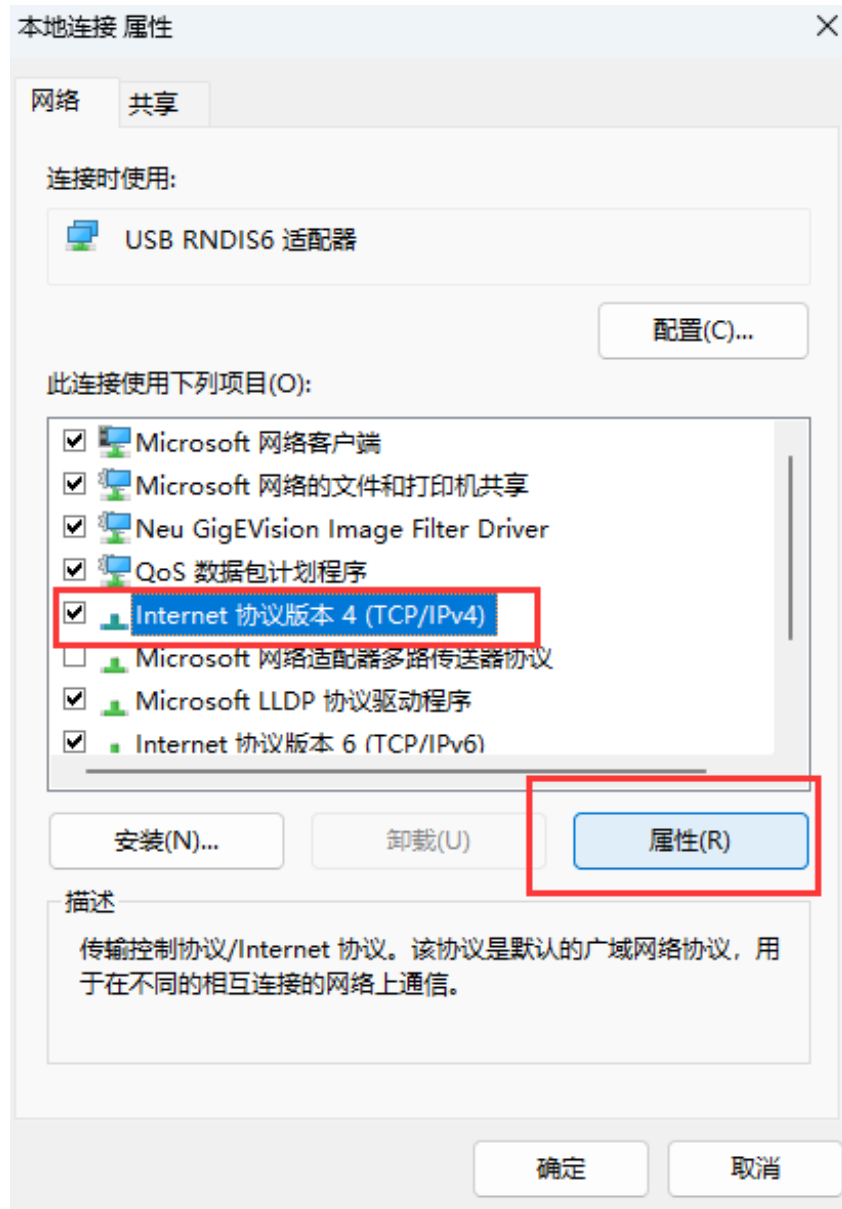


2) 配置 PC 接口 IP 地址

本步骤以 Windows 10 系统为例。

- ①在 PC 上打开“开始”，单击“设置”按钮，进入“Windows 设置”界面。
- ②选择“网络和 Internet”，单击“更改适配器选项”。
- ③鼠标右键单击“本地连接”后鼠标左键单击“属性”进入“本地连接 属性”界面
(使用 Type-C 接口连接时一般为“本地连接 x”，x 为数字，以实际显示的数字为准。

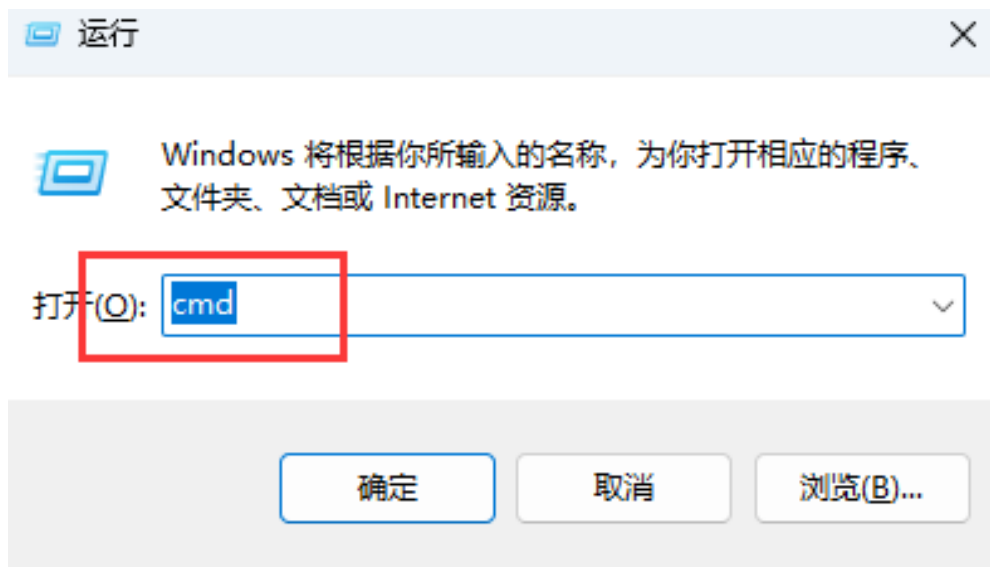
- ④选择“Internet 协议版本 4 (TCP/IPv4)”，单击“属性



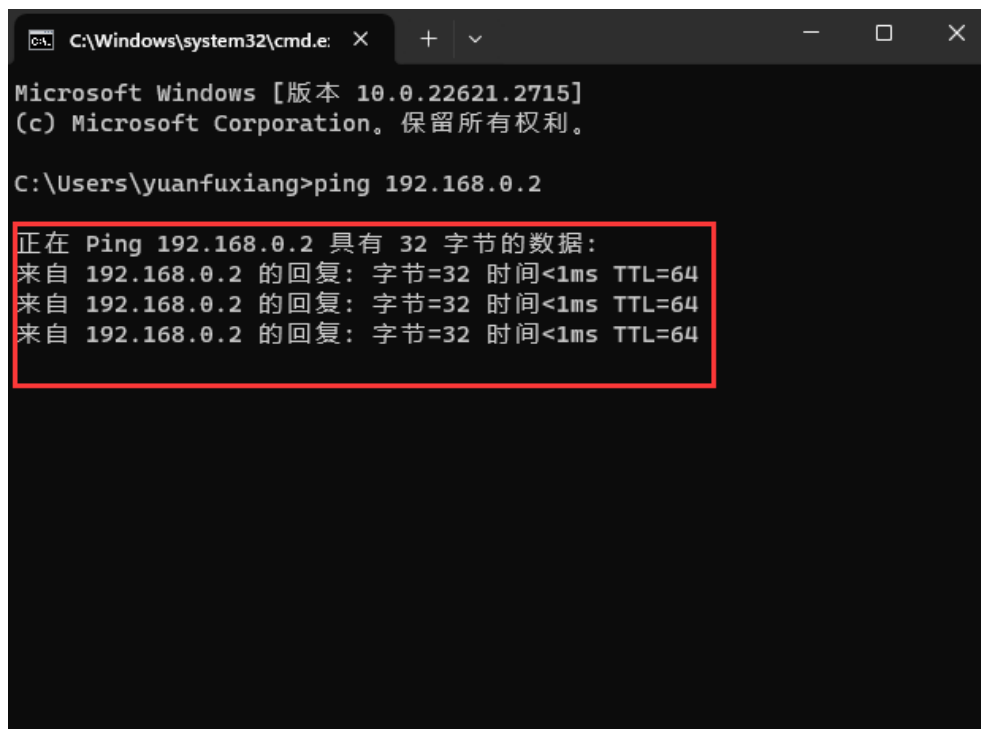
⑤勾选“使用下面的 IP 地址”选项，填写 IP 地址（图示以 192.168.0.101 为例）和子网掩码，默认网关与 DNS 服务器地址为空，单击“确定”保存。



⑥使用快捷键“Win+R”，在运行窗口输入 cmd 进入命令行窗口。



⑦输入 ping 192.168.0.2 命令，查看是否能够访问开发板 IP。



```
C:\Windows\system32\cmd.e  X  +  v  -  □  X

Microsoft Windows [版本 10.0.22621.2715]
(c) Microsoft Corporation。保留所有权利。

C:\Users\yuanfuxiang>ping 192.168.0.2

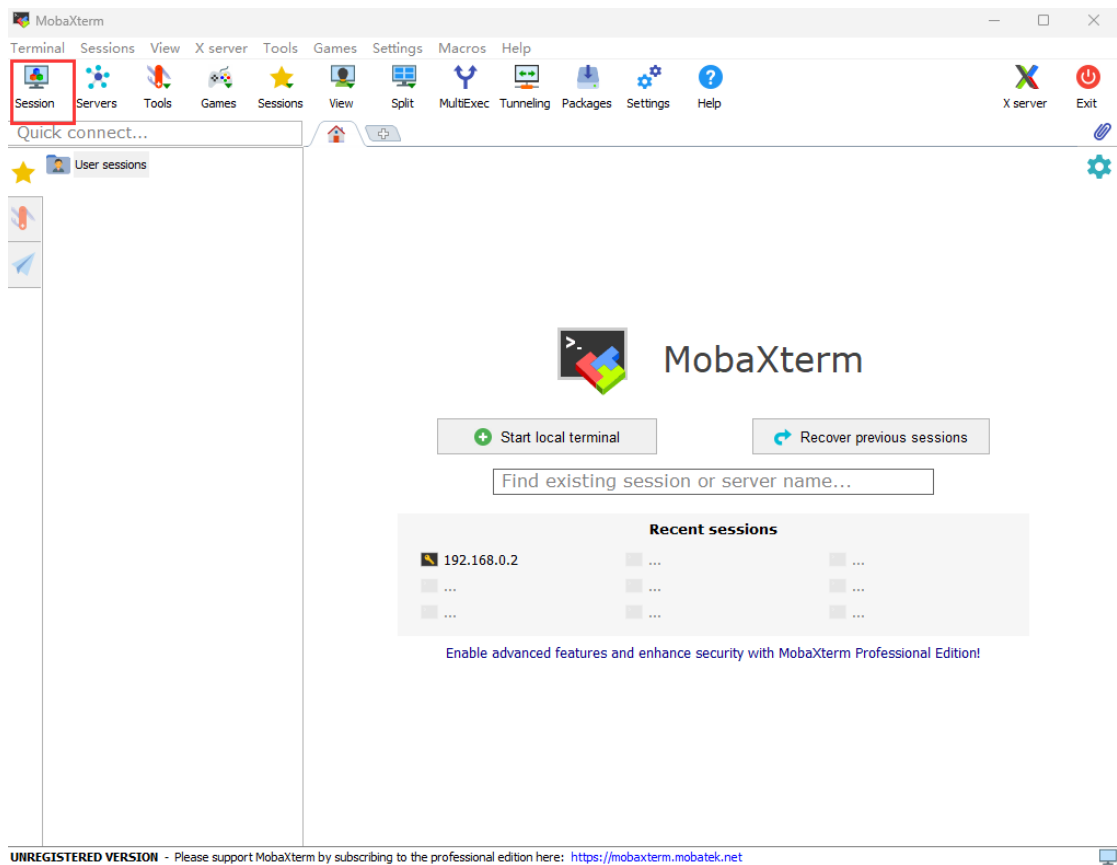
正在 Ping 192.168.0.2 具有 32 字节的数据:
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64
```

3) 使用 ssh 远程登录开发板

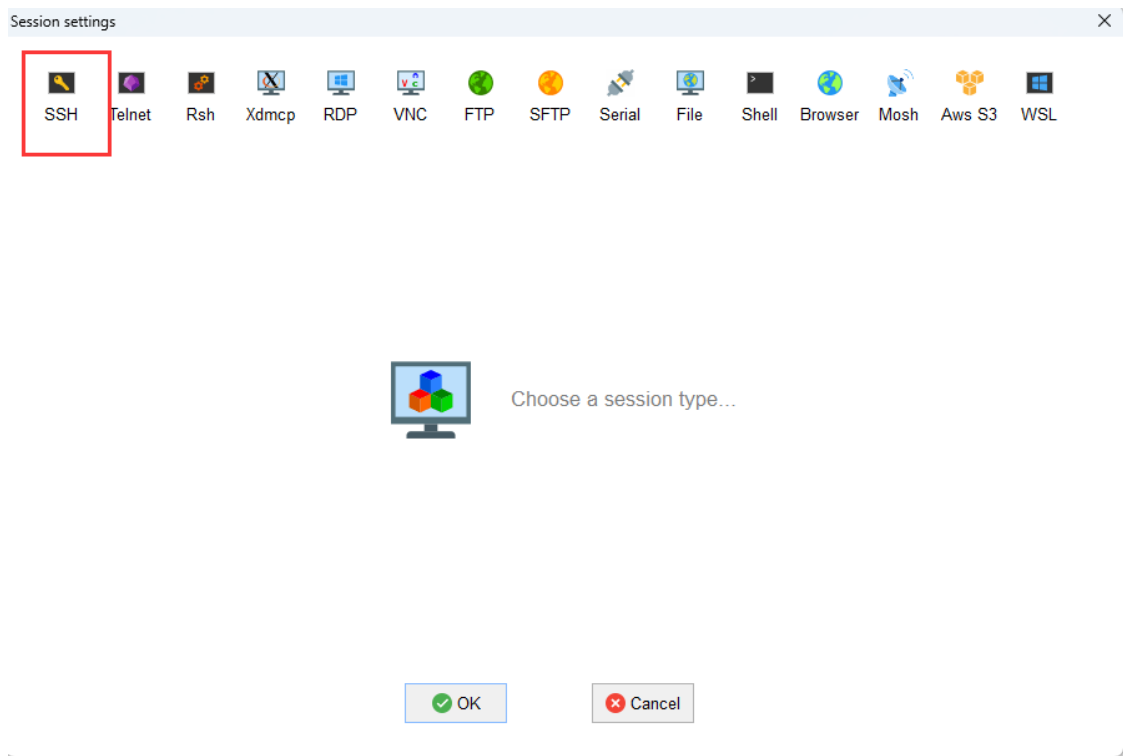
本文使用 MobaXterm 工具在 PC 上远程登录开发板，单击[下载链接](#)获取 MobaXterm 软件压缩包，解压获得“MobaXterm_Personal_22.2.exe”。

①双击“MobaXterm_Personal_22.2.exe”启动 SSH 登录工具。

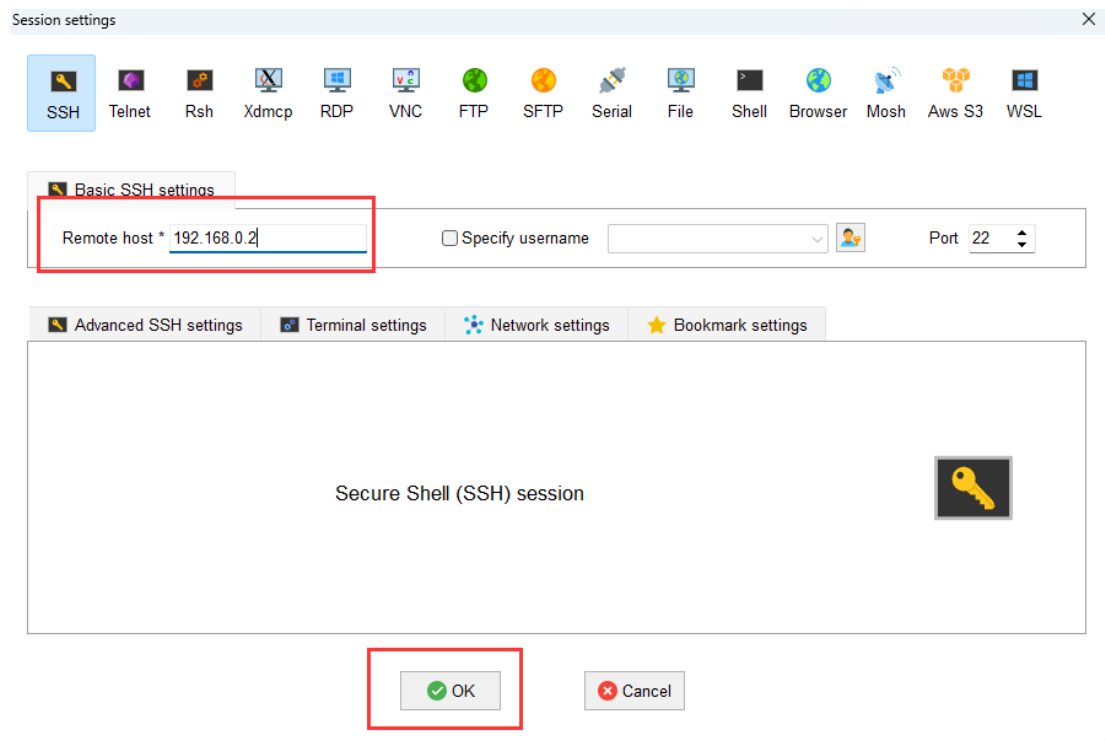
②单击左上方的“Session”进入界面。



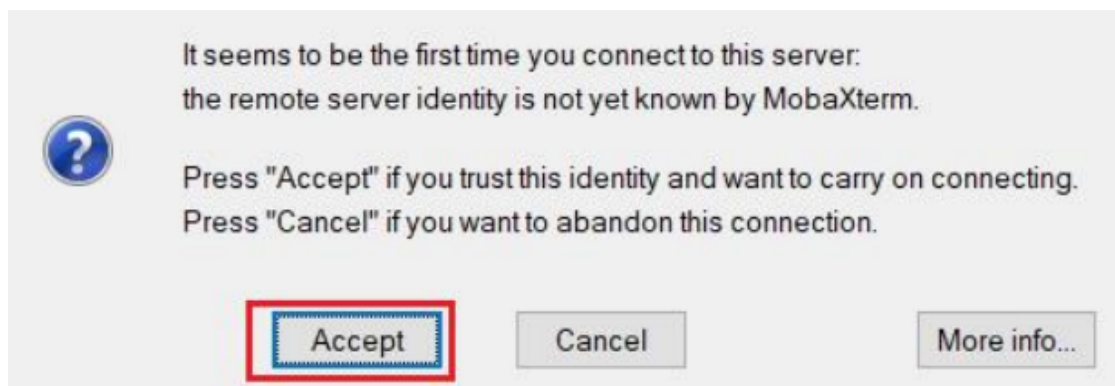
③单击左上方的“SSH”进入 SSH 连接配置界面



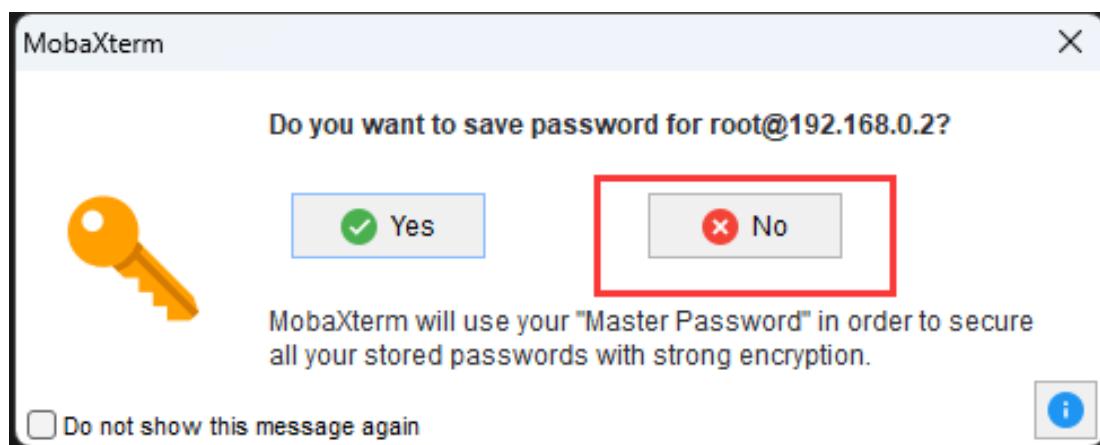
④根据硬件连接方式填写开发者套件连接 PC 的接口实际 IP 地址（一键制卡中配置的 IP 地址）。图示以 192.168.0.2 为例，请根据实际修改。



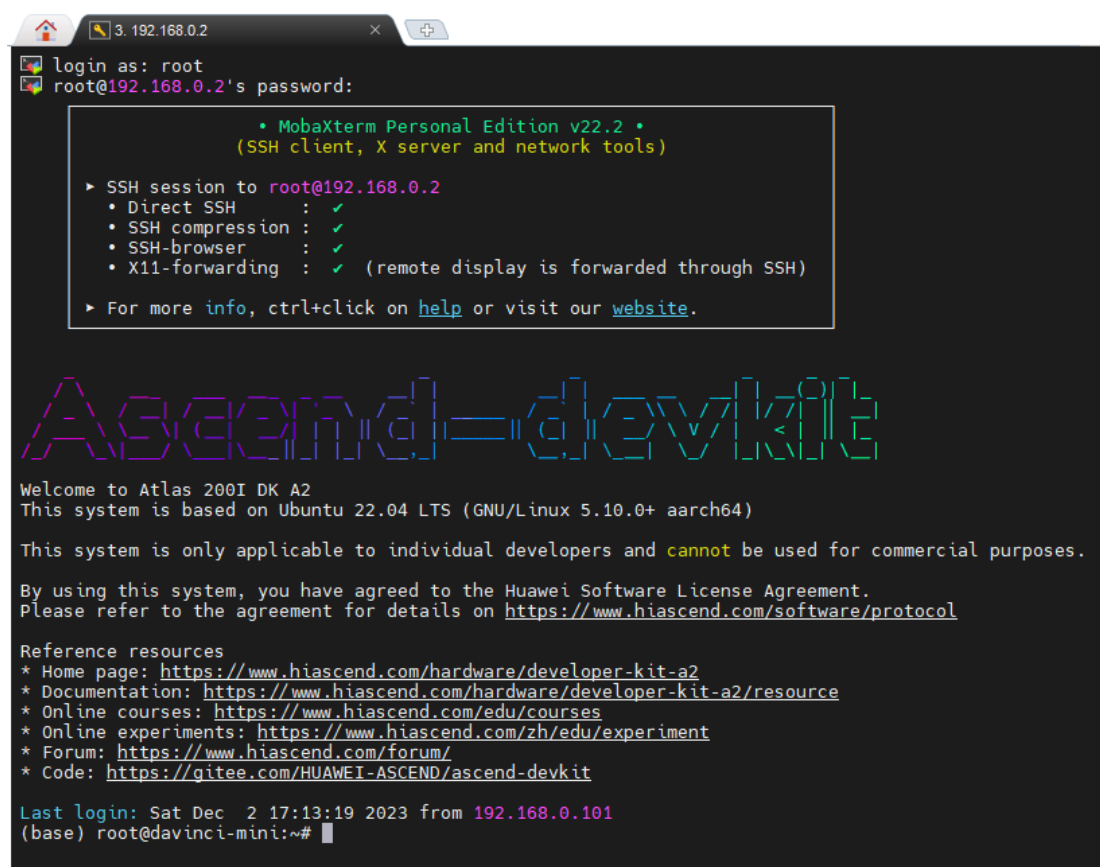
⑤单击“OK”按钮，首次连接开发者套件时，SSH 工具提示是否信任连接的设备，单击“Accept”。



⑥进入远程登录界面后，输入 root 用户名登录密码（默认为 Mind@123）登录开发者 套件，请修改默认密码，并妥善保管新密码。 SSH 工具界面会出现保存密码提示，可以单击“No”，不保存密码直接登录开发者套件。



⑦远程登录开发者套件成功界面如图所示。



4) 机器狗中继连接外网

注：本教程主要为暂时没有外置无线网卡的用户提供连接外网方案，教程以中继手机热点为例进行说明，中继无线路由器同理。

1、电脑连接机器狗的 wifi，点击电脑右下角的 wifi 图标，点击属性查看 IPv4 DNS 服务器的 ip 地址。如下图所示。



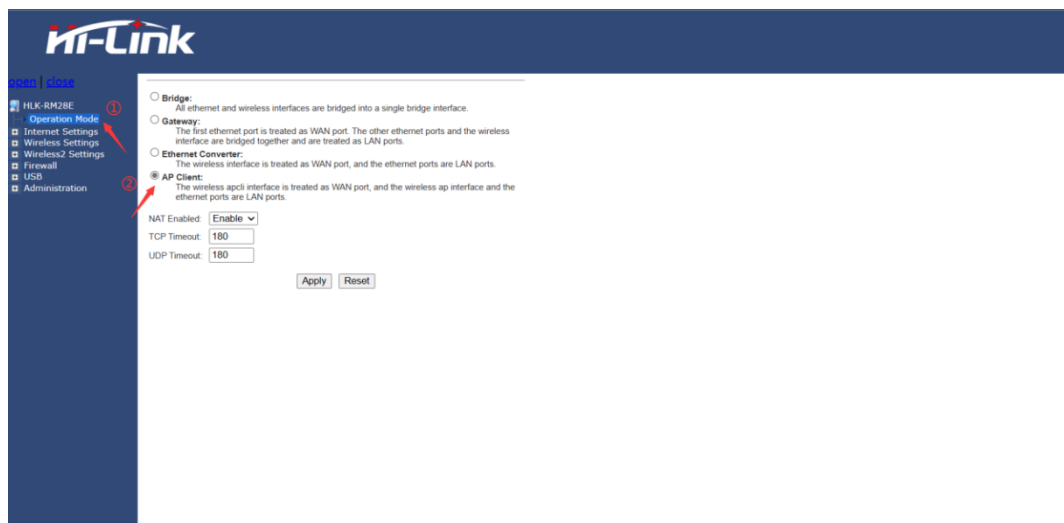
2、打开电脑浏览器，输入 IPv4 DNS 服务器的 ip，默认登录用户和密码均为 admin。如下图所示。



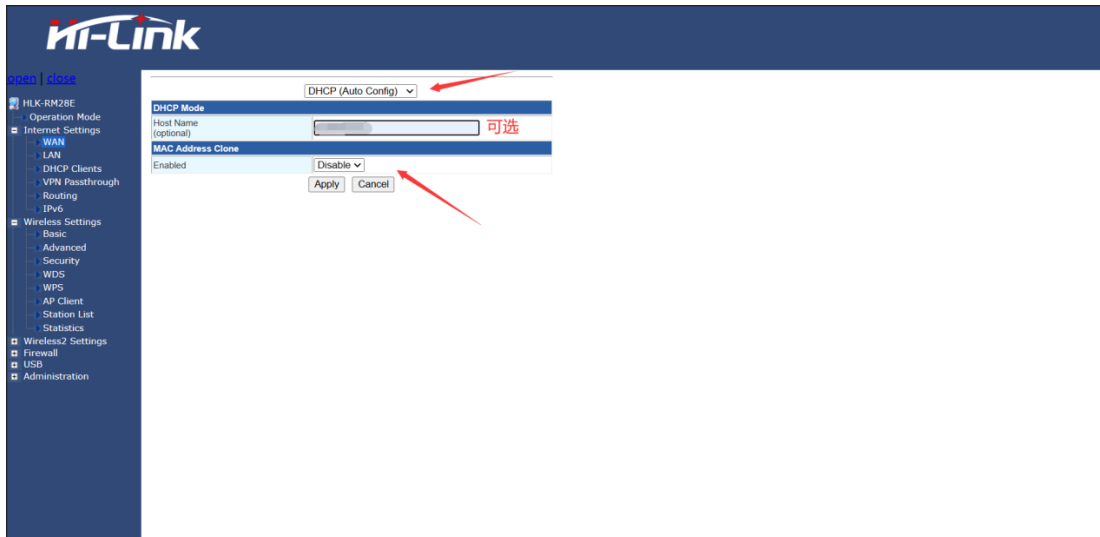
3、打开手机热点。

4、进入 Hi-link 设置页面，点击 Operation Mode，选择 AP Client，最后点击 Apply。

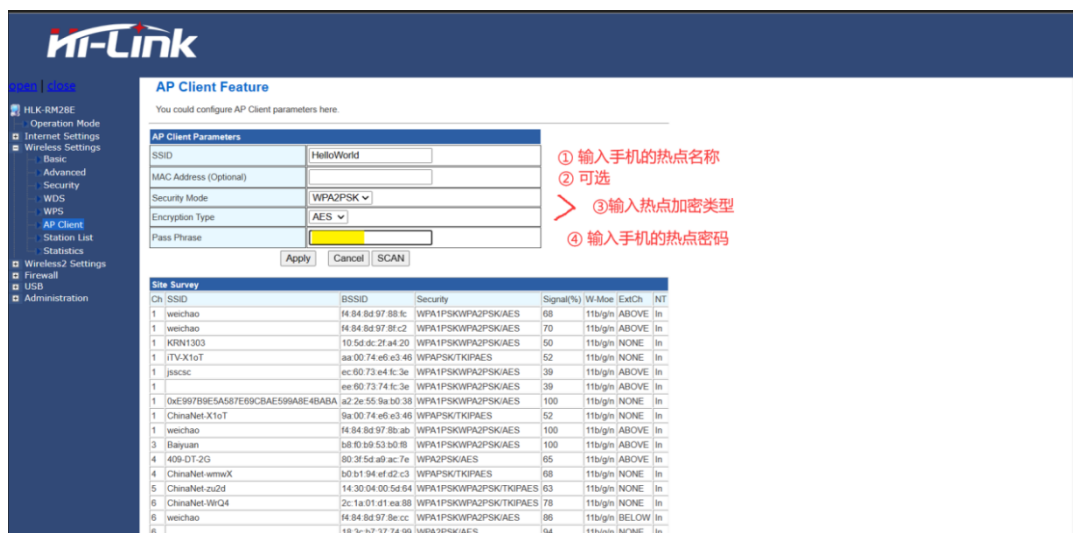
如下图所示。



5、展开 Internet Settings，点击 WAN，参照下图进行设置，最后点击 Apply。



6、展开 Wireless Settings，点击 AP Client，查看 site Survey 表中是否有自己的手机热点名称，如若没有点击 SCAN 进行扫描，在 AP client Parameters 表中参照下图进行设置，最后点击 Apply。



7、重启机器狗，注意中继时热点需一直处于开启状态，电脑连接机器狗的 wifi，使用远程软件登录，此时机器狗处于连接外网状态。测试是否有网络，点击浏览器，搜索任意内容，如“百度学术”，能正常显示下图说明正常。



3.2. 软件环境 (YOLO 训练)

(1) Pytorch 虚拟环境简介

Torch 是一个用于深度学习的数学计算库，而 Pytorch 则是一个基于 Torch 的 Python 机器学习库，可看作其提供了 Torch 应用于 Python 的接口。而搭建 Pytorch 虚拟环境，通俗地讲，就是搭建一个包含了 Pytorch 的相关包的 Python 解释器的环境，即是专门用于处理基于 Python 的深度学习的问题的环境。

(2) CUDA 简介

CUDA (Compute Unified Device Architecture) 中文名为统一计算设备架构，是显卡厂商 NVDIV 推出的运算平台，可以帮助 GPU 处理图形相关的计算问题。

我们可以通过在命令行输入以下命令来查看我们电脑的 CUDA 版本配置，下面可以看到我的 CUDA 版本为 11.7 (若电脑没有独立显卡，则该命令输入无效，独立显卡问题在第四部分说明，可先看该部分说明)。

(3) 开发板配置模型训练所需软件环境

①在开发板中已经安装了 miniconda3，我们可以直接使用 conda 命令安装虚拟环境。

```
Reference resources
* Home page: https://www.hiascend.com/hardware/developer-kit-a2
* Documentation: https://www.hiascend.com/hardware/developer-kit-a2/resource
* Online courses: https://www.hiascend.com/edu/courses
* Online experiments: https://www.hiascend.com/zh/edu/experiment
* Forum: https://www.hiascend.com/forum/
* Code: https://gitee.com/HUAWEI-ASCEND/ascend-devkit

Last login: Sat Dec 30 14:18:07 2023 from 192.168.0.101
(base) root@davinci-mini:~#
```

②使用 conda 命令创建虚拟环境

```
Last login: Sat Dec 30 14:18:07 2023 from 192.168.0.101
(base) root@davinci-mini:~# conda create -n chatbot python=3.7.16
```

```
conda create -n chatbot python=3.7.16
```

③在终端回弹中输入 y

The following packages will be downloaded:

package	build	
-----	-----	
_libgcc_mutex-0.1	main	2 KB
_openmp_mutex-5.1	51_gnu	1.4 MB
ca-certificates-2023.12.12	hd43f75c_0	126 KB
libffi-3.4.4	h419075a_0	139 KB
openssl-1.1.1w	h2f4d8fa_0	3.7 MB
pip-23.3.2	pyhd8ed1ab_0	1.3 MB conda-forge
python-3.7.16	h89984f6_0	44.8 MB
setuptools-68.2.2	pyhd8ed1ab_0	454 KB conda-forge
sqlite-3.41.2	h998d150_0	1.4 MB
wheel-0.38.4	py37hd43f75c_0	63 KB
xz-5.4.5	h998d150_0	663 KB
-----	-----	
Total:		54.0 MB

The following NEW packages will be INSTALLED:

_libgcc_mutex	pkgs/main/linux-aarch64::_libgcc_mutex-0.1-main
_openmp_mutex	pkgs/main/linux-aarch64::_openmp_mutex-5.1-51_gnu
ca-certificates	pkgs/main/linux-aarch64::ca-certificates-2023.12.12-hd43f75c_0
ld_impl_linux-aarch64	pkgs/main/linux-aarch64::ld_impl_linux-aarch64-2.38-h8131f1_0
libffi	pkgs/main/linux-aarch64::libffi-3.4.4-h419075a_0
libgcc-ng	pkgs/main/linux-aarch64::libgcc-ng-11.2.0-h1234567_1
libgomp	pkgs/main/linux-aarch64::libgomp-11.2.0-h1234567_1
libstdcxx-ng	pkgs/main/linux-aarch64::libstdcxx-ng-11.2.0-h1234567_1
ncurses	pkgs/main/linux-aarch64::ncurses-6.4-h419075a_0
openssl	pkgs/main/linux-aarch64::openssl-1.1.1w-h2f4d8fa_0
pip	conda-forge/noarch::pip-23.3.2-pyhd8ed1ab_0
python	pkgs/main/linux-aarch64::python-3.7.16-h89984f6_0
readline	pkgs/main/linux-aarch64::readline-8.2-h998d150_0
setuptools	conda-forge/noarch::setuptools-68.2.2-pyhd8ed1ab_0
sqlite	pkgs/main/linux-aarch64::sqlite-3.41.2-h998d150_0
tk	pkgs/main/linux-aarch64::tk-8.6.12-h241ca14_0
wheel	pkgs/main/linux-aarch64::wheel-0.38.4-py37hd43f75c_0
xz	pkgs/main/linux-aarch64::xz-5.4.5-h998d150_0
zlib	pkgs/main/linux-aarch64::zlib-1.2.13-h998d150_0

Proceed ([y]/n)? y

④虚拟环境创建成功如下所示:

```
Downloading and Extracting Packages

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#     $ conda activate chatbot
#
# To deactivate an active environment, use
#
#     $ conda deactivate
```

⑤进入刚刚创建的虚拟环境

```
(base) root@davinci-mini:~# conda activate chatbot
(chatbot) root@davinci-mini:~#
```

```
conda activate chatbot
```

⑥安装软件包 `pytorch==1.13.1`, `transformers==4.26.1`, `sentencepiece==0.1.97`, `numpy==1.21.6`, `Flask==2.2.5`, `Flask-Cors==4.0.0`, `tqdm==4.66.1`, `filelock==3.12.2`。
安装命令 `pip install xxx` (`xxx` 表示需要安装的软件包), 例如, 安装 `numpy` 的命令为:

```
(ks) C:\Users\>pip install numpy==1.22.4|
```

```
pip install numpy==1.22.4
```

注意: 安装 `pytorch` 的步骤为

⑦进入 `pytorch` 官网选择合适的命令安装 `pytorch`。此页面通常是最新版的 `pytorch`, 想要安装其他版本的可点击下面的按钮: [Previous versions of PyTorch](#)。

PyTorch

Get Started Ecosystem PyTorch Edge Blog Tutorials Docs Resources GitHub

INSTALL PYTORCH

Select your preferences and run the install command. Stable represents the most currently tested and supported version of PyTorch. This should be suitable for many users. Preview is available if you want the latest, not fully tested and supported, builds that are generated nightly. Please ensure that you have met the prerequisites below (e.g., **numpy**), depending on your package manager. Anaconda is our recommended package manager since it installs all dependencies. You can also [install previous versions of PyTorch](#). Note that LibTorch is only available for C++.

PyTorch Build	Stable (2.1.1)	Preview (Nightly)
Your OS	Linux	Mac Windows
Package	Conda	Pip LibTorch Source
Language	Python	C++ / Java
Compute Platform	CUDA 11.8	CUDA 12.1 ROCm 5.6 CPU
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118</pre>	

NOTE: PyTorch LTS has been deprecated. For more information, see [this blog](#).

[Previous versions of PyTorch](#)

QUICK START WITH CLOUD PARTNERS

Get up and running with PyTorch quickly through popular cloud platforms and machine learning services.

- Amazon Web Services
- Google Cloud Platform
- Microsoft Azure

⑧选择合适的命令进行运行，本文使用命令

```
(chatbot) root@davinci-mini:/# conda install pytorch==1.13.1 torchvision==0.14.1 torchaudio==0.13.1 -c pytorch
```

```
conda install pytorch==1.13.1 torchvision==0.14.1 torchaudio==0.13.1 -c pytorch
```

⑨本实验开发环境所选命令和安装成功提示如下所示。

```
Installing collected packages: typing-extensions, pillow, numpy, torch, torchvision, torchaudio
Successfully installed numpy-1.26.2 pillow-10.1.0 torch-1.8.2+cu102 torchaudio-0.8.2 torchvision-0.9.2+cu102 typing-extensions-4.8.0
```

3.3. 安装和配置 CUDA

1、先检查 pytorch 是否满足 cuda 的版本

Error! Unknown document property name.



```
1 import torch
2 print(torch.__version__)
3
4 print("Is CUDA available:", torch.cuda.is_available())
5 print("CUDA version:", torch.version.cuda)
```

Run 1 x

```
2.2.2+cpu
Is CUDA available: False
CUDA version: None
```

```
1.import torch

2.print(torch.__version__)

3.print("Is CUDA available:",torch.cuda.is_available())
```

torch 的版本是 cpu 版本， 且不支持 cuda

2、 cuda Toolkit 的网站: <https://developer.nvidia.com/cuda-toolkit-archive>

在官网下载对应的 cuda 和 cuda Toolkit， 安装成功之后会在环境变量中显示下面图

片：

系统变量(S)	
变量	值
CUDA_PATH	C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4
CUDA_PATH_V12_4	C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4

```

C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4\bin
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4\libnvvp
C:\Program Files (x86)\Common Files\MVS\Runtime\Win32_i86
C:\Program Files (x86)\Common Files\MVS\Runtime\Win64_x64
%SystemRoot%\system32
%SystemRoot%
%SystemRoot%\System32\Wbem
%SYSTEMROOT%\System32\WindowsPowerShell\v1.0\
%SYSTEMROOT%\System32\OpenSSH\
D:\Microsoft VS Code\bin
D:\Git\cmd
D:\ProgramData\anaconda3
D:\ProgramData\anaconda3\Library\bin
D:\ProgramData\anaconda3\Library\mingw-w64\bin
C:\Program Files\NVIDIA Corporation\Nsight Compute 2024.1.1\
C:\Program Files (x86)\NVIDIA Corporation\PhysX\Common
C:\Program Files\NVIDIA Corporation\NVIDIA NvDLISR
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4\include

```

在 PATH 中也能看到，注意红色的地方是没有自动生成的，需要手动添加，在 cmd 中也会显示

```

C:\WINDOWS\system32\cmd.
Microsoft Windows [版本 10.0.22631.3296]
(c) Microsoft Corporation. 保留所有权利。

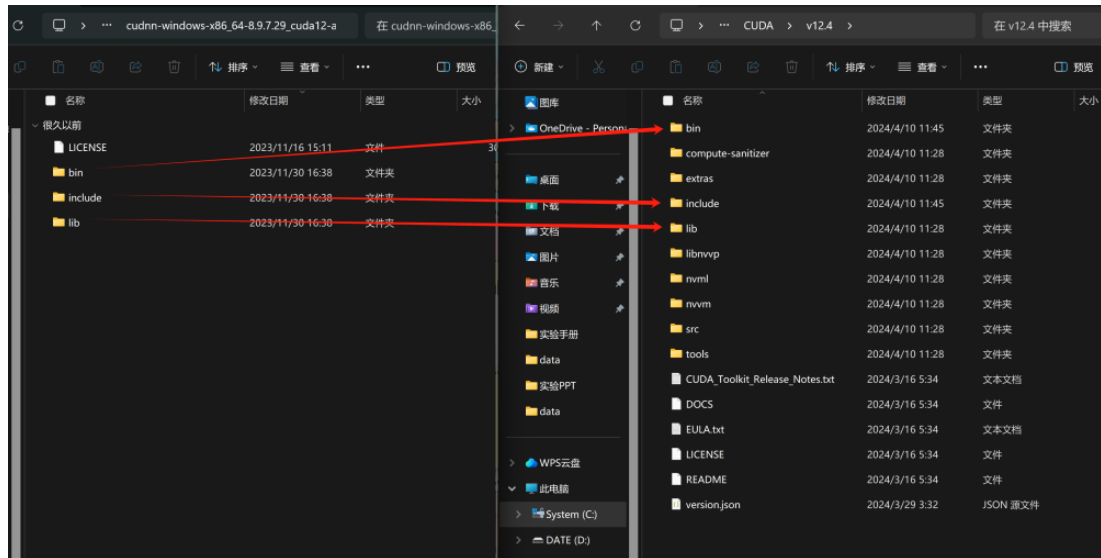
C:\Users\15210>nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Thu_Mar_28_02:30:10_Pacific_Daylight_Time_2024
Cuda compilation tools, release 12.4, V12.4.131
Build cuda_12.4.r12.4/compiler.34097967_0

```

```
nvcc -V
```

3、再安装 CUDNN 对应的版本: <https://developer.nvidia.com/rdp/cudnn-archive>

下载对应的 CUDNN 和 cuda 版本之后，解压，然后将 CUDNN 中的文件全部复制到 cuda 中对应的文件



根据步骤来，复制完文件之后，进去这个目录，运行下面两个脚本

```
C:\Windows\System32\cmd.e x + v
Microsoft Windows [版本 10.0.22631.3296]
(c) Microsoft Corporation. 保留所有权利。
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4\extras\demo_suite>deviceQuery.exe
deviceQuery.exe Starting...

  CUDA Device Query (Runtime API) version (CUDA static linking)

Detected 1 CUDA Capable device(s)

Device 0: "NVIDIA GeForce RTX 3050 Ti Laptop GPU"
  CUDA Driver Version / Runtime Version      12.4 / 12.4
  CUDA Capability Major/Minor version number: 8.6
  Total amount of global memory:              4096 MBytes (4294443008 bytes)
  (20) Multiprocessors, (128) CUDA Cores/MP: 2560 CUDA Cores
  GPU Max Clock rate:                        1485 MHz (1.49 GHz)
  Memory Clock rate:                         6001 Mhz
  Memory Bus Width:                          128-bit
  L2 Cache Size:                             2097152 bytes
  Maximum Texture Dimension Size (x,y,z)     1D=(131072), 2D=(131072, 65536), 3D=(16384, 16384, 16384)
  Maximum Layered 1D Texture Size, (num) layers 1D=(32768), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers 2D=(32768, 32768), 2048 layers
  Total amount of constant memory:            zu bytes
  Total amount of shared memory per block:    zu bytes
  Total number of registers available per block: 65536
  Warp size:                                 32
  Maximum number of threads per multiprocessor: 1536
  Maximum number of threads per block:        1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
```

deviceQuery.exe

Error! Unknown document property name.

```
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4\extras\demo_suite>bandwidthTest.exe
[CUDA Bandwidth Test] - Starting...
Running on...

Device 0: NVIDIA GeForce RTX 3050 Ti Laptop GPU
Quick Mode

Host to Device Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)      Bandwidth(MB/s)
33554432                   10992.5

Device to Host Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)      Bandwidth(MB/s)
33554432                   12092.3

Device to Device Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)      Bandwidth(MB/s)
33554432                   146545.3

Result = PASS

NOTE: The CUDA Samples are not meant for performance measurements. Results may vary when GPU Boost is enabled.
```

bandwidthTest.exe

这样说明 CUDNN 安装成功

4、在 pycharm 中测试

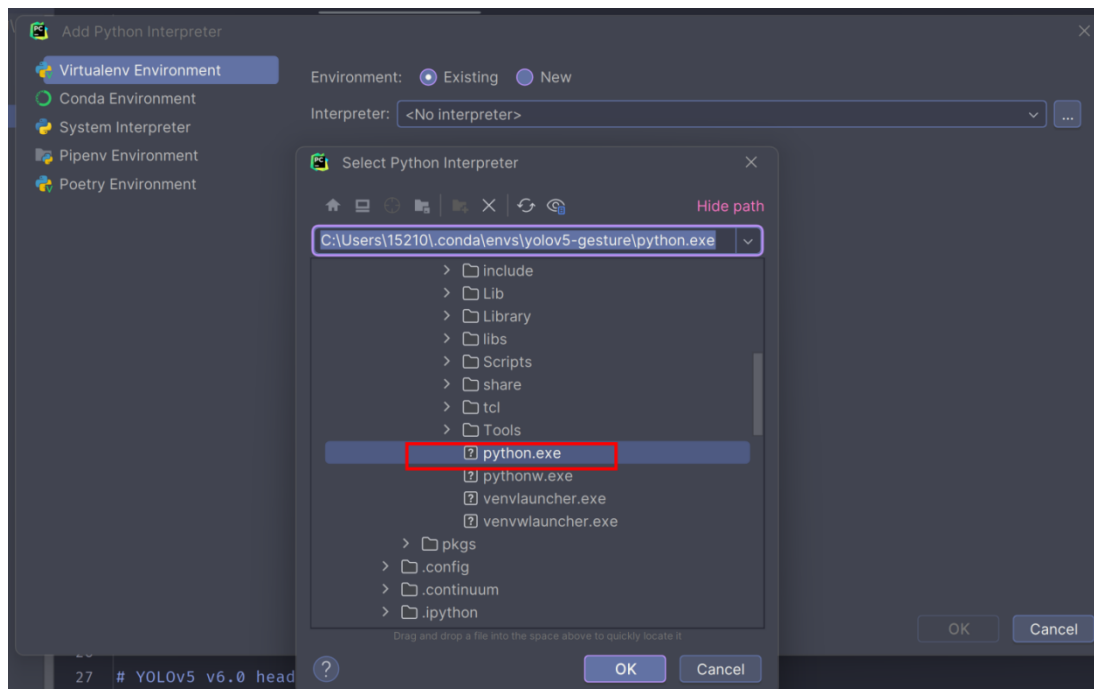
```
import torch
print(torch.__version__)
print(torch.cuda.is_available())

3 x
:
C:\Users\15210\.conda\envs\yolov5-
2.2.2+cu121
True
```

- 1.import torch
- 2.print(torch.__version__)
- 3.print(torch.cuda.is_available())

True 说明成功，cu121 是指 cuda 版本 12.1

选择 Add Local Interpreter——>Load Environments——>选择自己的环境



选择 OK，虚拟环境配置成功。

3.4.开发板环境（机器狗网址配置）

用户可通过 SSH 远程连接到运动主机。

1. 将开发主机连接到机器人 WiFi。
2. 在开发主机上打开 SSH 连接软件，输入 `ssh firefly@192.168.1.120`，密码为 `firefly`，即可远程连接运动主机；
3. 进入 `jy_exe`，并打开配置文件：

```
cd /home/firefly/jy_exe/conf/  
vim network.toml
```

Error! Unknown document property name.

4. 配置文件 `network.toml` 内容如下：

```
ip = '192.168.1.102'  
  
target_port = 43897  
  
local_port = 43893  
  
~
```

5. 修改配置文件第一行中的 IP 地址，使得代码能够接收到机器狗数据：

- 如果代码在机器人运动主机内运行，IP 设置为运动主机 IP：192.168.1.120；
- 如果代码在用户自己的开发主机中运行，设置为开发主机的静态 IP：192.168.1.xxx；

6. 重启运动程序使配置生效。

```
cd /home/firefly/jy_exe  
  
sudo ./stop.sh  
  
sudo ./restart.sh
```

第四章 工程文件目录结构

4.1.PC

这个章节存放的内容主要作用是采集图片数据，对数据进行采集，处理，建模。

名称	修改日期	类型	大小
.github	2024/4/23 11:55	文件夹	
.idea	2024/7/3 14:18	文件夹	
__pycache__	2024/5/10 14:08	文件夹	
classify	2024/4/23 11:55	文件夹	
data	2024/4/23 11:55	文件夹	
dataset	2024/5/14 14:54	文件夹	
models	2024/5/8 16:03	文件夹	
runs	2024/7/3 14:16	文件夹	
segment	2024/4/23 11:55	文件夹	
utils	2024/5/14 14:47	文件夹	
.dockerignore	2022/11/22 7:23	DOCKERIGNORE...	4 KB
.gitattributes	2022/11/22 7:23	Git Attributes 源...	1 KB
.gitignore	2022/11/22 7:23	Git Ignore 源文件	4 KB
.pre-commit-config.yaml	2022/11/22 7:23	Yaml 源文件	2 KB
Arial.ttf	2024/4/24 10:24	TrueType 字体文件	756 KB
benchmarks.py	2022/11/22 7:23	Python 源文件	8 KB
CONTRIBUTING.md	2022/11/22 7:23	Markdown 源文件	5 KB
detect.py	2024/5/10 11:44	Python 源文件	14 KB
export.py	2022/11/22 7:23	Python 源文件	31 KB
hubconf.py	2022/11/22 7:23	Python 源文件	8 KB

PC 端 yolov5 的文件夹，主要内容为 **dataset** 中存放的数据文件，其他的地方不需要修改。**train.py** 是主要的训练文件。

4.2. Ascend

```
(base) root@davinci-mini:/opt/lab/SmartLogistics# tree -I "__pycache__|*.pyc" -L 10
.
├── camera
│   ├── HKcamera.py
│   └── det_utils.py
├── command
│   ├── __init__.py
│   └── udp_command.py
├── getAndSavePicture.py
├── robotstatuswatcher
│   ├── __init__.py
│   └── listener.py
├── sendcommand
│   ├── SendToCommand.py
│   ├── __init__.py
│   └── heartbeat.py
├── smart_logistics_main.py
├── smart_logistics_models
│   ├── 1.om
│   ├── fusion_result.json
│   ├── model.onnx
│   └── predefined_classes.txt
├── socketnetwork
│   ├── __init__.py
│   └── network_utils.py
├── speeds
│   ├── __init__.py
│   └── sportspeed.py
├── threading_utils
│   └── ThreadTemplates.py
```

开发板的文件夹总名叫做‘SmartLogistics’，如图所示，其中command文件中的udp_command.py文件的用来存放指令码，sendcommand中的heartbeat.py是发送心跳包，SendToCommand.py是发送指令，socketnetwork中的network_utils.py是与机械狗建立通信的脚本，speeds中的sportspeed.py文件是机械狗运动的速度方程，threading_utils中的ThreadTemplates.py文件是线程，camera中放的是摄像头的使用文件，robotstatuswatcher中使用来监听机器狗状态的，smart_logistics_models中存放的是模型文件，getAndSavePicture.py是采集数据的脚本。

4.3. ModelArts



yolov5 与 PC 端相似，output 用来存放训练好的模型。

第五章 数据工程

5.1. 配置开发板的 HK 驱动

下载并安装 MVS 驱动文件（Linux）



海康威视官网下载链接：

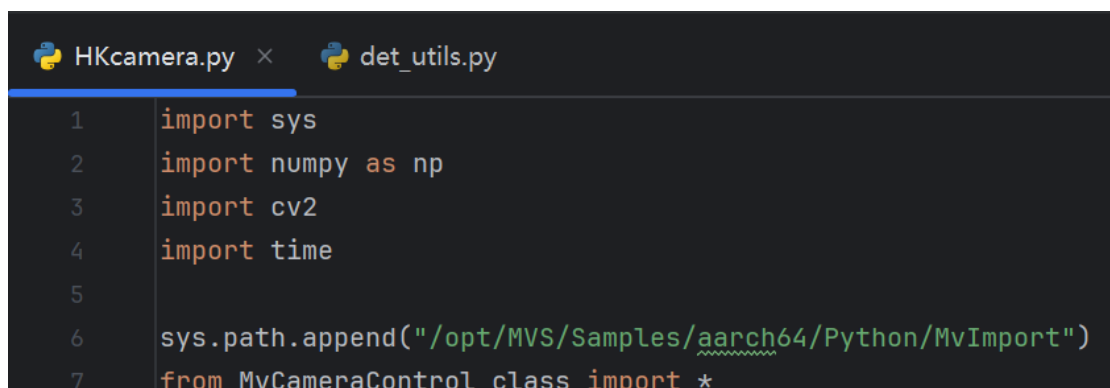
<https://www.hikrobotics.com/cn/machinevision/service/download?module=0>



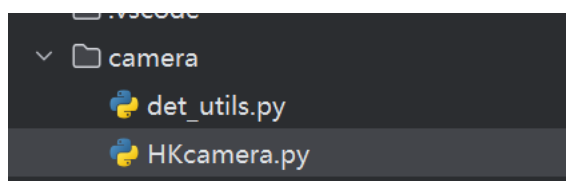
打开 MobaXterm 远程控制，进入开发板安装驱动指令：`dpkg -i MVS-3.0.1_aarch64_20240422.deb`

在代码中使用需要加入包的路径：

```
sys.path.append("/opt/MVS/Samples/aarch64/Python/MvImport")from MvCameraControl_class
import*
```



使用摄像头的代码：`python HKcamera.py`



Error! Unknown document property name.

5.2. 编写数据采集脚本

getAndSavePicture.py

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. import cv2
5. import time
6.
7. import sys
8. sys.path.append('camera/')
9. from HKcamera import getImage
10. from det_utils import get_labels_from_txt, letterbox, scale_coords, nms
    , draw_bbox # 模型前后处理相关函数
11.
12. # 导入定义的机械狗相关的包
13. from threading_utils.ThreadTemplates import * # 线程的模板与基
    本轴指令的发送
14. from sendcommand import SendToCommand, heartbeat # 发送简单指令与
    心跳包
15. from socketnetwork import network_utils # 通信函数
16. from command.udp_command import * # 存放各种结构体
    与状态数据、指令
```

```

17.from speeds.sportspeed import *                                # 运动精准控制的
    基础版模型

18.from robotstatuswatcher.listener import *                      # 监听机械狗状态

19.from HandDistance.ridge_np import *                            # 测量距离

20.#getImage 从 HKcamera 模块导入，用于获取图像。
    get_labels_from_txt, letterbox, scale_coords, nms, draw_bbox 从
    det_utils 模块导入，这些函数通常用于

21.#目标检测模型的前后处理。导入与机械狗相关的包，如线程模板、发送指令、网络通
    信、命令结构体、运动控制和状态监听等。

22.def save_image(img):

23.    image_name = f'train_images/demo/{time.time()}.jpg'

24.    cv2.imwrite(image_name, img)

25.    print(f"图像已保存为: {image_name}")

26.

27.print("按 's' 保存图像，按 'q' 退出程序。")

28.

29.# 主循环

30.#通过 getImage() 获取图像。

31.#使用 cv2.imshow 显示图像。

32.#使用 cv2.waitKey(1) 检测按键事件，如果按下 's'，则调用 save_image 保存图
    像；如果按下 'q'，则退出循环。

33.while True:

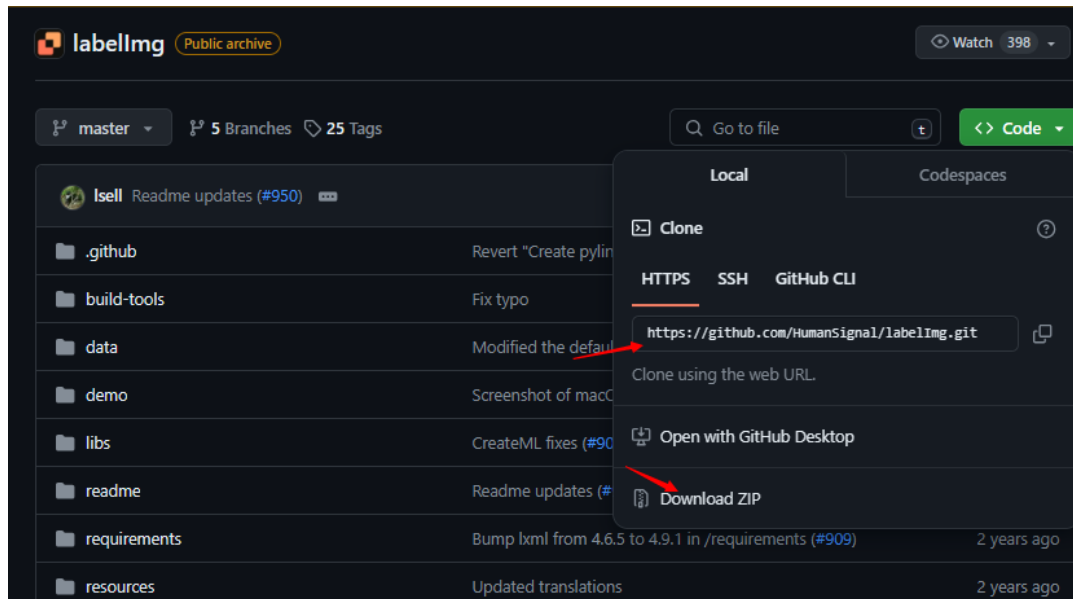
34.    img = getImage() # 获取图像

```

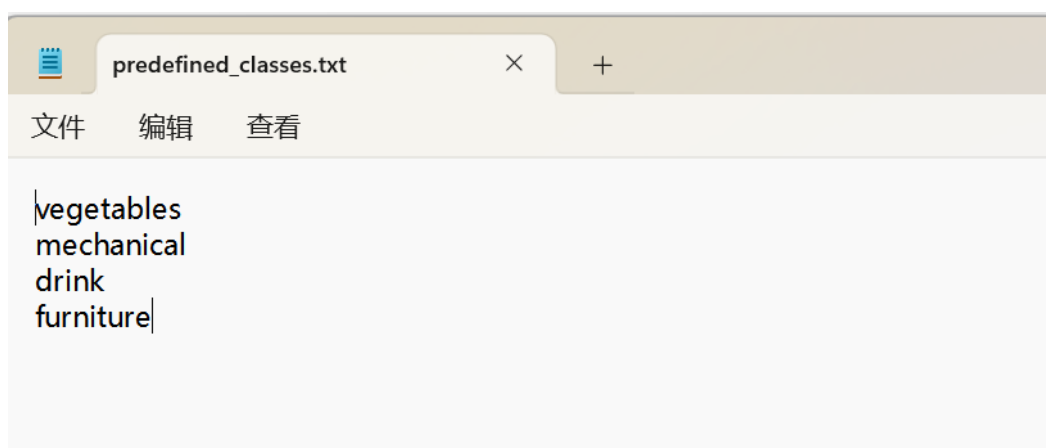
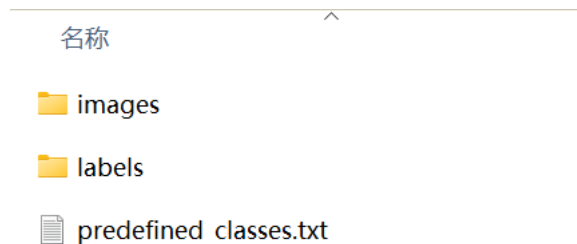
```
35.  
36.    cv2.imshow('Camera Feed', img) # 显示图像  
37.    key = cv2.waitKey(1) & 0xFF # 检测按键事件  
38.  
39.    if key == ord('s'):  
40.        save_image(img) # 保存图像  
41.    elif key == ord('q'):  
42.        break # 退出循环  
43.  
44. # 清理工作  
45. cv2.destroyAllWindows()
```

5.3. 配置 PC 端数据标注环境

下载数据标注工具 labelimg : <https://github.com/HumanSignal/labelImg>

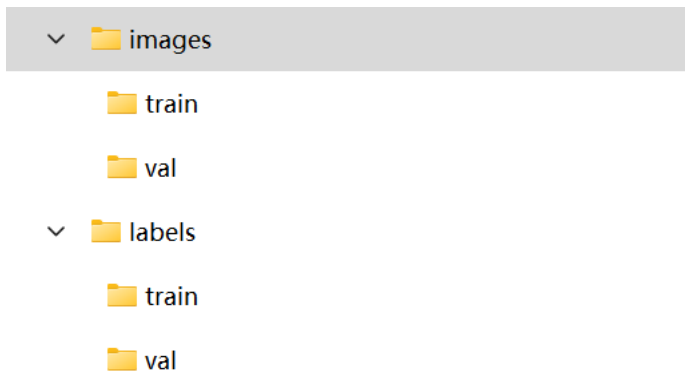


建立两个文件夹,分别是 images 用来存放训练集 train 的图片和验证集 val 的图片,labels 用来存放标注完 train 中对应的图片数据标签和 val 中的对应的图片数据标签,以及 predefined_classes.txt 存放标签类别。



/images/train 对应 labelImg 中的存放目录,而/labels/train 对应 labelImg 的改变存放目录,用于存放标注完图片的数据标签。

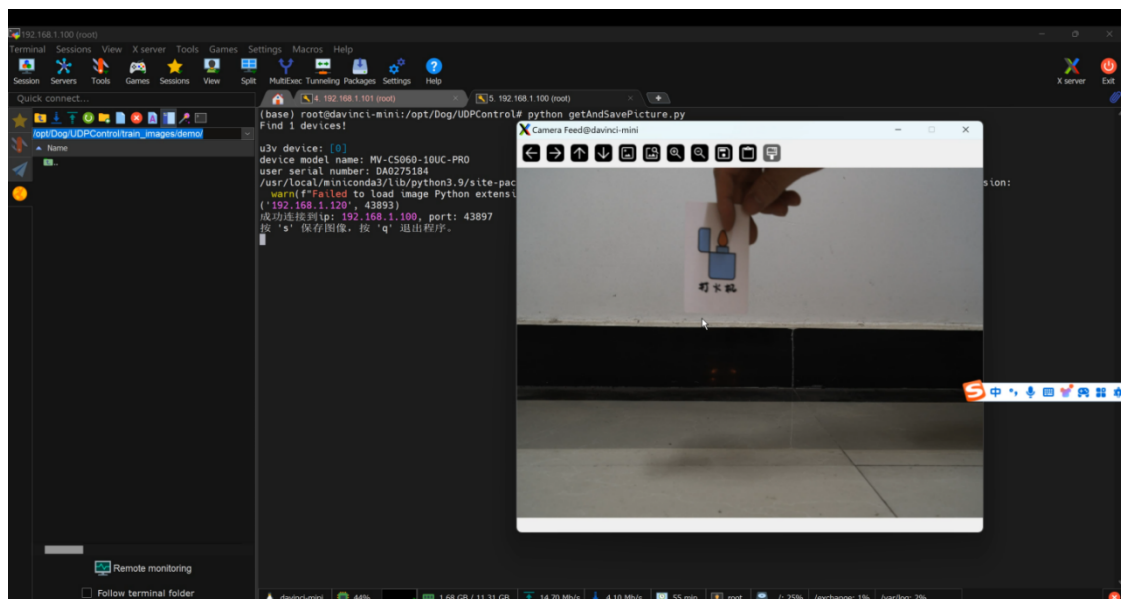
Error! Unknown document property name.



5.4. 标注数据集

5.4.1. 采集数据

1. 连接开发板，运用脚本采集机器狗视角的图像



2. 进入 pc 存放 labelImg 的目录中，运行 python labelImg.py



3. 新建一个 data.yaml，将数据集设置为项目中的相对路径

```

train.py  data.yaml x
1  # YOLOv5 🚀 by Ultralytics, GPL-3.0 license
2  # COCO128 dataset https://www.kaggle.com/ultralytics/coco128 (first 128 images)
3  # Example usage: python train.py --data coco128.yaml
4  # parent
5  # └─ yolov5
6  # └─ datasets
7  # └─ coco128 ← downloads here
8
9  # 这个文件是数据配置文件，存放着 数据集源路径root、训练集、验证集、测试集地址，类别个数，类别名称
10
11
12 # 数据集源路径root、训练集、验证集、测试集地址
13 train: dataset/images/train # root下的训练集地址 128 images
14 val: dataset/images/val # root下的验证集地址 128 images
15 test: # root下的验证集地址 128 images
16
17 # 数据集类别信息
18 nc: 4 # 数据集类别数量
19 names: ['vegetables', 'mechanical', 'drink', 'furniture'] # class names

```

在 yolov5.yaml，将 nc 改为 4，表示数据集中有四个类别

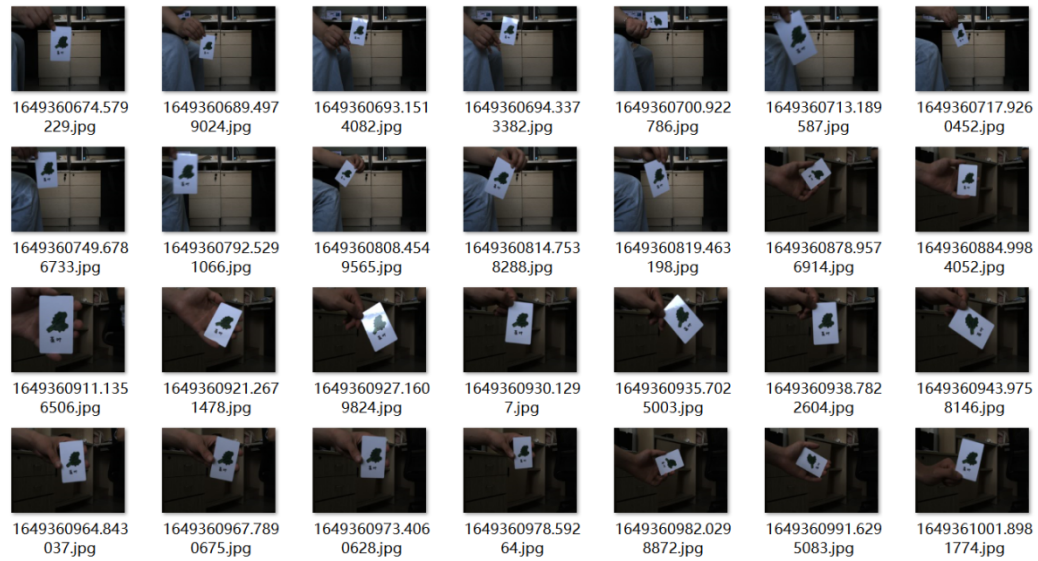
```

train.py  data.yaml  yolov5s.yaml x
1  # YOLOv5 🚀 by Ultralytics, GPL-3.0 license
2
3  # Parameters
4  nc: 4 # number of classes

```

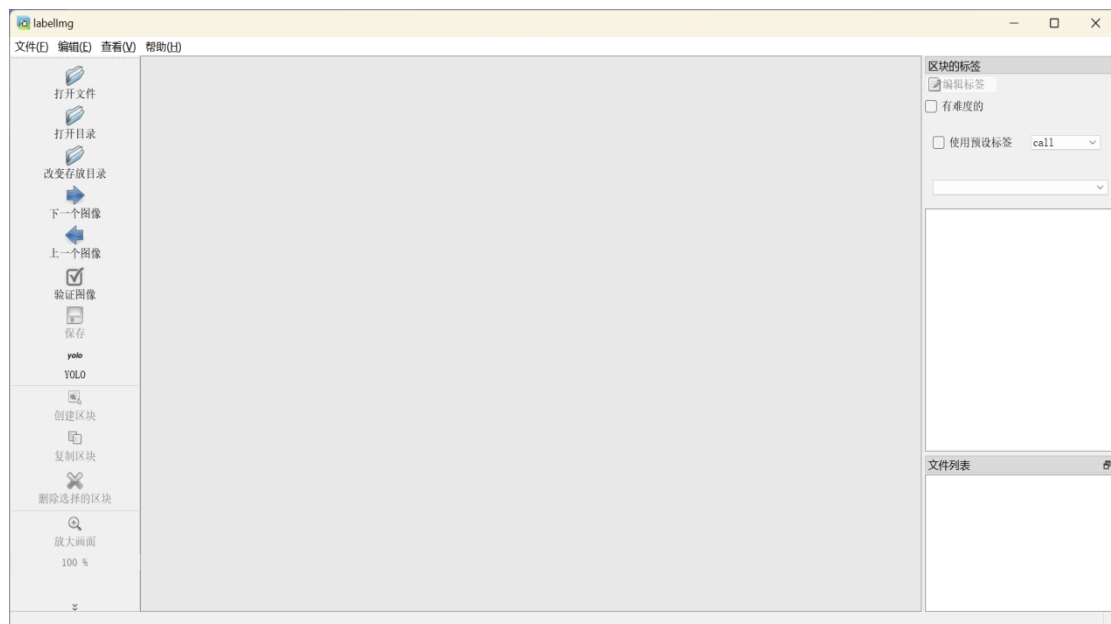
在采集的四类智慧物流产品中，以其中一类 vegetables 为例，可看到如下图所示。

Error! Unknown document property name.



5.4.2. 使用 labelImg 进行数据集标注

- 1、打开 anaconda 命令行界面
- 2、激活我们已有的环境，如果没有就先创建再激活
- 3、输入 labelImg 回车，打开标注界面，下图我对各个按钮进行了详细解释



4、同时在 predefined_classes.txt 中，设置所分类的标签名称；在 data 文件夹中分别新建 images、label 文件夹，同时在 images 文件夹中，新建 train 文件夹，里面存放训练集智慧物流产品照片，与之对应应在 label 文件夹中，新建 train 文件夹，里面用来存放标注好的数据集信息；同时在 images 文件夹中，再新建 val 文件夹，用来存放验证集物流产品照片，与之对应应在 label 文件夹中，新建 val 文件夹，里面用来存放标注好的数据集信息

- 5、在“打开目录”中打开刚刚的 data/images/train 文件夹，在“改变存放目录”中更换

保存路径为 data/label/train 文件夹

5.4.3. 数据标定

进入 labelImg-master 所在文件夹地址

```
(base) PS E:\> cd E:\Desktop\亿生为\labelImg-master
(base) PS E:\Desktop\亿生为\labelImg-master> python labelImg.py
E:\Desktop\亿生为\labelImg-master\labelImg.py:73: DeprecationWarning: sipPyTypeDict() is deprecated, the extension module should use sipPyTypeDictRef() instead
class MainWindow(QMainWindow, WindowMixin):
[('palm', [(207, 279), (310, 279), (310, 406), (207, 406)], None, None, False)]
```

选择好“打开目录”即图片所在位置、“改变存放目录”即标签存放位置



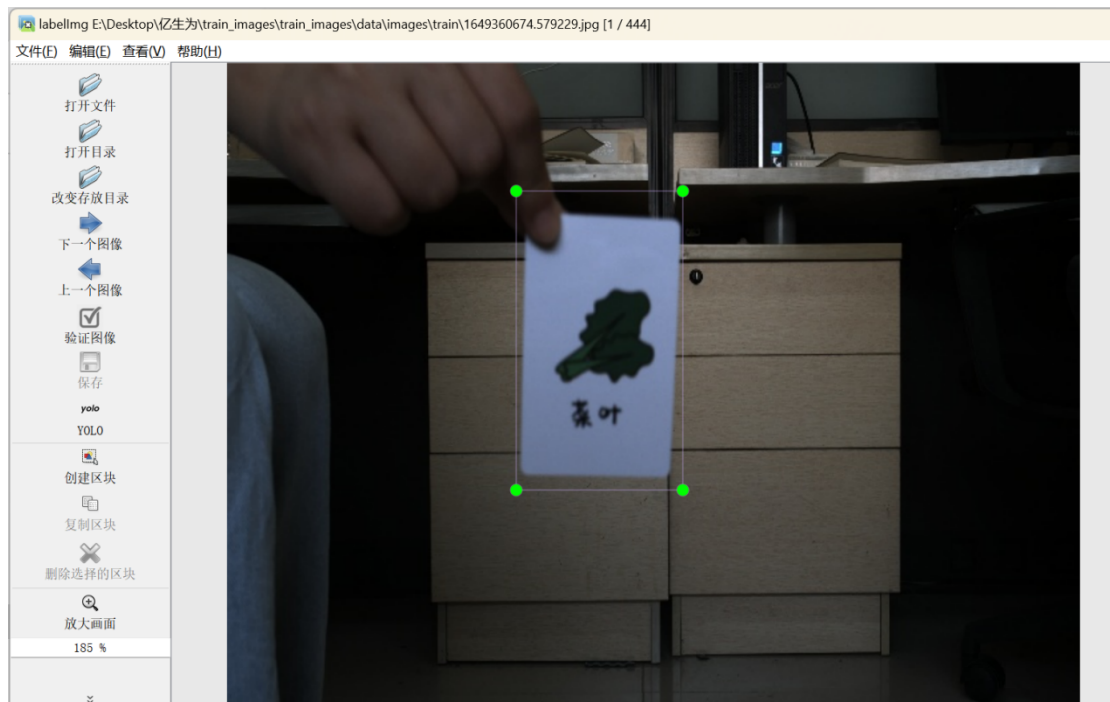
后输入 python labelImg.py 进入数据标注页面，点击”创造区块“，然后出现十字架，选择对应类别：vegetables，进行拖动框住标注区域

然后输入标签，点击 palm，再点击左侧的“保存”标签

在 data/label/train 里面可以看到我们的标签，在此介绍 labelImg 的三种标签格式：

- (1) PascalVOC 标签格式，保存为 xml 文件
- (2) YOLO 标签格式，保存为 txt 文件
- (3) CreateML 标签格式，保存为 json 文件

点击“下一个图像”继续标注下一张



5.4.4. 数据预览

数据标注完成后，若标注图片是 `\labeldata\images\train` 中，则可到 `\labeldata\labels\train` 中查看标签文件，`images` 中存放训练集 `train` 图，`labels` 中存放训练集 `train` 所对应的数据标签 `txt` 文件，可看到其文件名对应图片名称，`txt` 中的四列数据分别对应数据标注区块各个含义，具体如下图所示。



第六章 模型本地化训练与推理



6.1. 训练流程

1. 模型训练

- a. 详细流程：调用 YOLOv5 训练文件，读取制作的数据集，来训练我们的模型。
- b. 实验注意点：超参数的设置，以及数据集文件结构。
- c. 重点知识：YOLOv5 训练过程，超参数设置。
- d. 完成本步骤后输出：在 runs/train/exp 文件夹下保存的 pt 模型。

2. 模型测试

- a. 详细流程：调用 detect.py 文件的参数，对模型进行验证，
- b. 实验注意点：需要指定测试的图片和 pt 模型路径。
- c. 重点知识：检测验证。
- d. 完成本步骤后输出：在指定的目录中保存检测后的图片。

3. 模型转换

- a. 详细流程：在 PC 机上将 .pt 模型转换为 .onnx 模型；再在 PC 上将 onnx 模型转换为 om 模型。
- b. 实验注意点：om 模型转换命令参数。
- c. 重点知识：YOLOv5 转换 onnx 模型流程；onnx 模型转换为 om 模型的命令使用。
- d. 完成本步骤后输出：可以在昇腾开发板上推理的 .om 模型。

4. 开发板相机设置

- a. 详细流程：打开工业相机保护盖，连接工业相机和昇腾开发板，调整 HKcamera.py 代码参数和补光灯亮度，使工业相机清晰成像。
- b. 实验注意点：HKcamera.py 代码内容调整后，保存后才能生效。
- c. 重点知识：昇腾开发板上工业相机的调整。
- d. 完成本步骤后输出：运行 HKcamera.py 代码后显示清晰图片。

5. 开发板推理

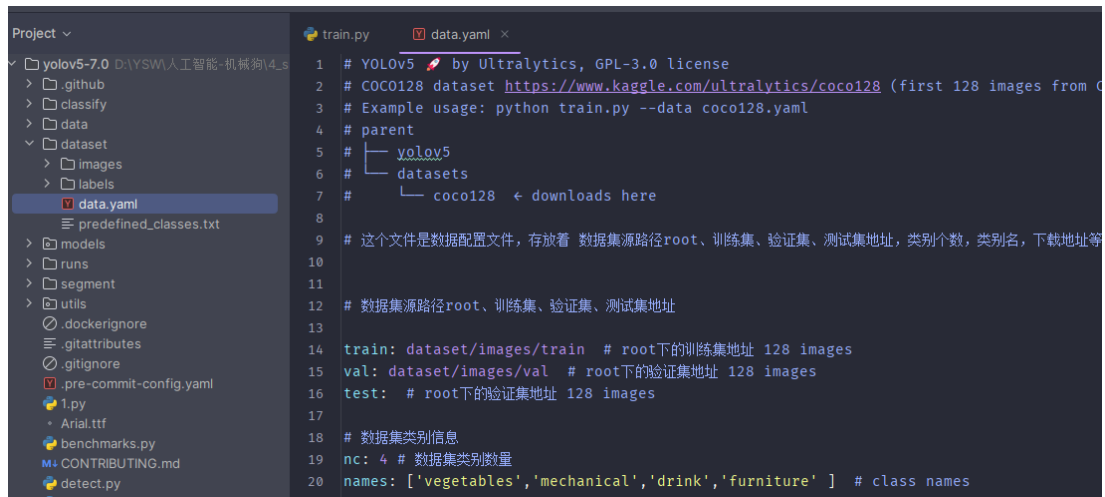
- a. 详细流程：编写昇腾开发板推理 .om 模型的代码。
- b. 实验注意点：代码中 MindX SDK 调用时需要方便拓展。
- c. 重点知识：昇腾开发板推理 .om 的方法
- d. 完成本步骤后输出：运行代码后会输出目标检测后的结果图。

6.2. 训练参数详细介绍与修改

参数通常用于配置训练深度学习模型时的各种选项。这些参数允许用户在训练模型时灵活地设置各种配置选项。下面是对这些参数的解释：

- weights: 指定初始权重的路径，默认为 ROOT/yolov5s.pt。
- cfg: 指定模型配置文件的路径，默认为空，需要用户指定。
- data: 指定数据集配置文件的路径，默认为 ROOT/dataset/data.yaml。
- train_url: 用于 ModelArts 平台的训练 URL，默认为 "weights"。
- hyp: 指定超参数配置文件的路径，默认 ROOT/data/hyps/hyp.scratch-low.yaml。
- epochs: 指定总的训练周期数，默认为 100。
- batch-size: 指定所有 GPU 的总批量大小，默认为 16，-1 表示自动批量。
- imgsz 或 --img 或 --img-size: 指定训练和验证时的图像大小（像素），默认为 640。
- rect: 如果设置，将使用矩形训练。
- resume: 如果设置，将恢复最近的训练。
- nosave: 如果设置，只保存最终的检查点。
- noval: 如果设置，只在最后一个周期进行验证。
- noautoanchor: 如果设置，将禁用自动锚点调整。
- noplots: 如果设置，不保存任何绘图文件。
- evolve: 指定用于进化超参数的代数，默认为 300。
- bucket: 指定用于 gsutil 的存储桶。
- cache: 指定图像缓存的方式，可以是 'ram' 或 'disk'，默认为 'ram'。
- image-weights: 如果设置，将使用加权图像选择进行训练。
- device: 指定使用的设备，例如 '0' 或 '0,1,2,3' 表示 GPU 设备号，或者 'cpu' 表示使用 CPU。

1.新建一个 dataset 文件夹，在里面除了存放数据之外再新建一个 data.yaml，对数据集进行配置



1. train: dataset/images/train

2. val: dataset/images/val

3. test:

4.

5. # number of classes

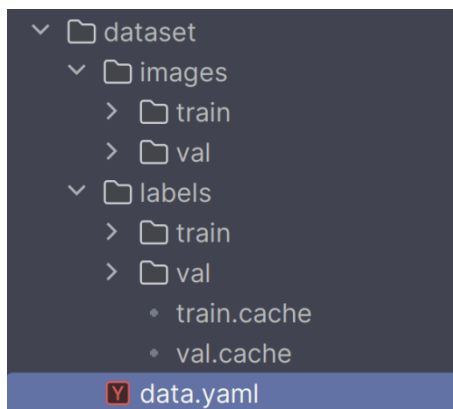
6. nc: 2

7.

8. # class names

9. names: ['vegetables','mechanical',drink,'furniture']

2.整理数据集文件结构，确定和如下图示相同。并且 labels 下需要存放 classes.txt 文件。



3.修改 train.py 文件的参数，其中主要包括--weights[预训练权重路径]、--cfg[模型结构配

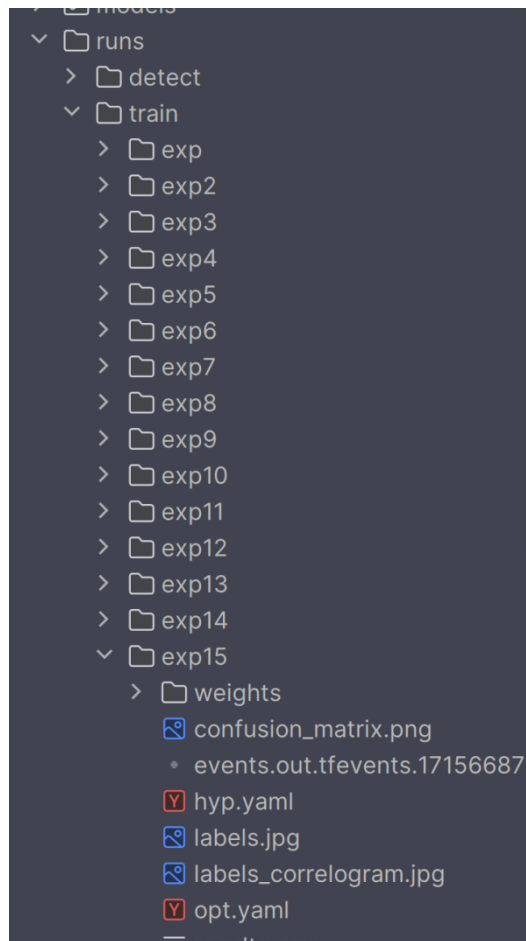
Error! Unknown document property name.

置]、--data[数据集配置文件]、--epochs[训练轮数]、--batch-size[批数据大小]。

本文所采取的参数如下：

```
1. python train.py --weights yolov5s.pt --cfg yolov5s.yaml --  
   data dataset/data.yaml --epochs 100 --batch-size 4
```

4.运行 train.py，等到训练完成后就可以在 runs 文件下的 train 目录找到训练的结果。



6.3. 模型训练

本文的参数选择: `python train.py --img 640 --batch 16 --epoch 500 --data dataset/data.yaml -
-cfg models/yolov5s.yaml --weights yolov5s.pt`

```
Anaconda Powershell Prompt x + v
4      -1 2    115712 models.common.C3      [128, 128, 2]
5      -1 1    295424 models.common.Conv [128, 256, 3, 2]
6      -1 3    625152 models.common.C3      [256, 256, 3]
7      -1 1    1180672 models.common.Conv [256, 512, 3, 2]
8      -1 1    1182720 models.common.C3      [512, 512, 1]
9      -1 1    656896 models.common.SPPF [512, 512, 5]
10     -1 1    131584 models.common.Conv [512, 256, 1, 1]
11     -1 1      0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
12     [-1, 6] 1      0 models.common.Concat [1]
13     -1 1    361984 models.common.C3      [512, 256, 1, False]
14     -1 1    33024 models.common.Conv [256, 128, 1, 1]
15     -1 1      0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
16     [-1, 4] 1      0 models.common.Concat [1]
17     -1 1     90880 models.common.C3      [256, 128, 1, False]
18     -1 1    147712 models.common.Conv [128, 128, 3, 2]
19     [-1, 14] 1      0 models.common.Concat [1]
20     -1 1    296448 models.common.C3      [256, 256, 1, False]
21     -1 1    590336 models.common.Conv [256, 256, 3, 2]
22     [-1, 10] 1      0 models.common.Concat [1]
23     -1 1    1182720 models.common.C3      [512, 512, 1, False]
24     [17, 20, 23] 1 24273 models.yolo.Detect [4, [[10, 13, 16, 30, 33, 23], [30, 61, 62, 45], 59, 119], [116, 90, 156, 198, 373, 326]], [128, 256, 512]]
YOLOv5s summary: 214 layers, 7030417 parameters, 7030417 gradients, 16.0 GFLOPs

Transferred 342/349 items from yolov5s.pt
AMP: checks passed
optimizer: SGD(lr=0.01) with parameter groups 57 weight(decay=0.0), 60 weight(decay=0.0005), 60 bias
train: Scanning D:\YSW\人工智能-机械狗\4_smart_logistics\pc\yolov5-7.0\dataset\labels\train...: 0% | 0/444 00
```

```
Anaconda Powershell Prompt x + v
0/499    3.4G  0.09832  0.02916  0.04758  19      640: 100% | 28/28 00:10
Class    Images  Instances  P      R      mAP50  mAP50-95: 100% | 3/3 00:02
all      81      81      0.00333 0.925  0.0376  0.0103

Epoch   GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
1/499   4.02G  0.06039  0.0262   0.04308   25         640: 100% | 28/28 00:06
Class    Images  Instances  P      R      mAP50  mAP50-95: 100% | 3/3 00:01
all      81      81      0.0393  0.876  0.216  0.0497

Epoch   GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
2/499   4.02G  0.05452  0.023   0.04218   18         640: 100% | 28/28 00:06
Class    Images  Instances  P      R      mAP50  mAP50-95: 100% | 3/3 00:01
all      81      81      0.0995  0.359  0.0929  0.0378

Epoch   GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
3/499   4.02G  0.05215  0.01995  0.0409   24         640: 100% | 28/28 00:06
Class    Images  Instances  P      R      mAP50  mAP50-95: 100% | 3/3 00:01
all      81      81      0.0966  0.298  0.0879  0.0237

Epoch   GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
4/499   4.02G  0.05054  0.01744  0.03952   21         640: 100% | 28/28 00:06
Class    Images  Instances  P      R      mAP50  mAP50-95: 100% | 3/3 00:01
all      81      81      0.207  0.589  0.273  0.137

Epoch   GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
5/499   4.02G  0.04394  0.01362  0.03937   34         640: 46% | 13/28 00:03
```

训练完成之后在 runs\train\exp17 的文件夹下可以看到训练的文件

Results saved to runs\train\exp17

6.4. 模型测试

首先在 yolov5-7.0\runs\train\exp15\weight\1.png 中放入测试图片



修改 detect.py 文件的参数，对模型进行验证，其中主要包括--weights[训练权重路径]、--source[测试图片目录]、--data[数据集配置文件]

本文所采取的参数如下：

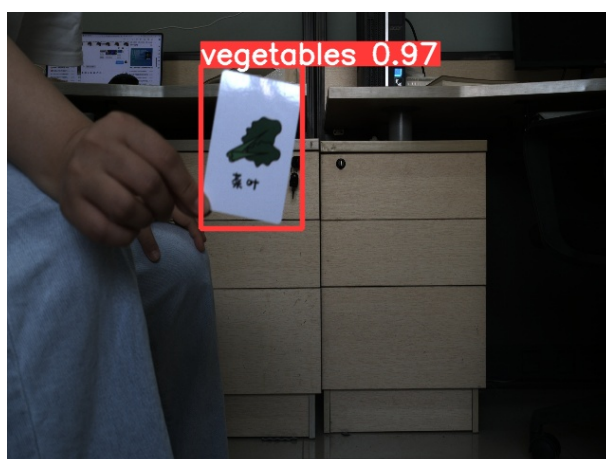
```
python detect.py --weights runs/model.pt --source "D:/YSW/ 人 工 智 能 - 机 械 狗
/4_smart_logistics/pc/yolov5-7.0/runs/1649360693.1514082.jpg" --data dataset/data.yaml
```

运行成功之后

```
(yolov5-gesture) PS D:\YSW\人工智能-机械狗\4_smart_logistics\pc\yolov5-7.0> python detect.py --weights runs/model.pt --source "D:/YSW/人工智能-机械狗/4_smart_logistics/pc/yolov5-7.0/runs/1649360693.1514082.jpg" --data dataset/data.yaml
detect: weights=['runs/model.pt'], source=D:/YSW/-/4_smart_logistics/pc/yolov5-7.0/runs/1649360693.1514082.jpg, data=dataset/data.yaml, imgsz=[640, 640], conf_thres=0.25, iou_thres=0.45, max_det=1000, device=, view_img=False, save_txt=False, save_conf=False, save_crop=False, nosave=False, classes=None, agnostic_nms=False, augment=False, visualize=False, update=False, project=runs\detect, name=exp, exist_ok=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False, vid_stride=1
YOLOv5 2024-5-14 Python-3.9.19 torch-2.2.2+cu121 CUDA:0 (NVIDIA GeForce RTX 3050 Ti Laptop GPU, 4096MiB)

Fusing layers...
YOLOv5s summary: 157 layers, 7020913 parameters, 0 gradients, 15.8 GFLOPs
image 1/1 D:\YSW\-\4_smart_logistics\pc\yolov5-7.0\runs\1649360693.1514082.jpg: 480x640 1 vegetables, 90.0ms
Speed: 0.0ms pre-process, 90.0ms inference, 120.0ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs\detect\exp10
(yolov5-gesture) PS D:\YSW\人工智能-机械狗\4_smart_logistics\pc\yolov5-7.0> |
```

结果保存在了下面路径 runs\detect\exp10



可以看到已经识别成功。

6.5. 模型转换

1. 首先将模型转换为 onnx

```
1. python export.py --weights runs/model.pt --include onnx
```

输入之后会看到有

```
(yolov5-gesture) PS D:\YSW\人工智能-机械狗\4_smart_logistics\pc\yolov5-7.0> python export.py --weights runs/model.pt --include onnx
export: data=D:\YSW\人工智能-机械狗\4_smart_logistics\pc\yolov5-7.0\data\coco128.yaml, weights=['runs/model.pt'], imgsz=[640, 640], batch_size=1, device=cpu, half=False, inplace=False, keras=False, optimize=False, int8=False, dynamic=False, simplify=False, opsset=12, verbose=False, workspace=4, nms=False, agnostic_nms=False, topk_per_class=100, topk_all=100, iou_thres=0.45, conf_thres=0.25, include=['onnx']
YOLOv5 2024-5-14 Python-3.9.19 torch-2.2.2+cu121 CPU

Fusing layers...
YOLOv5s summary: 157 layers, 7020913 parameters, 0 gradients, 15.8 GFLOPs
```

等待片刻会出现转换成功的提示，以及保存的位置

```
ONNX: starting export with onnx 1.16.0...
ONNX: export success 47.6s, saved as runs\model.onnx (27.2 MB)

Export complete (48.1s)
Results saved to D:\YSW\人工智能-机械狗\4_smart_logistics\pc\yolov5-7.0\runs
Detect:      python detect.py --weights runs\model.onnx
Validate:    python val.py --weights runs\model.onnx
PyTorch Hub: model = torch.hub.load('ultralytics/yolov5', 'custom', 'runs\model.onnx')
Visualize:   https://netron.app
```

2. 将得到的 ONNX 模型文件，修改名字为 model.onnx（方便区别，如果名字是 model 就不需要修改），上传到开发板

```
(base) root@davinci-mini:/opt# ll
total 27888
drwxr-xr-x  2 root root    4096 Apr 12 17:10 ./
drwxr-xr-x 24 root root    4096 Apr  8 2022 ../
-rw-r--r--  1 root root 28545135 Apr 12 17:10 model.onnx
```

运行下面命令

```
1. atc --model=model.onnx --framework=5 --output=model --soc_version=Ascend310B4
```

若提示如下信息，则说明模型转化成功。

ATC run success

```
(base) root@davinci-mini:/opt# atc --model=model.onnx --framework=5 --output=model --soc_version=Ascend310B4
ATC start working now, please wait for a moment.
...
ATC run success, welcome to the next use.
```

```
(base) root@davinci-mini:/opt# ll
total 42844
drwxr-xr-x  2 root root    4096 Apr 12 17:26 ./
drwxr-xr-x 24 root root    4096 Apr  8 2022 ../
-rw-r----- 1 root root    3022 Apr 12 17:26 fusion_result.json
-rw----- 1 root root 15307128 Apr 12 17:26 model.om
-rw-r--r-- 1 root root 28545135 Apr 12 17:10 model.onnx
```

第七章 在线 ModelArts 训练与模型转换及应用

7.1. 自定义训练教程——物流分拣识别

7.1.1. 使用预训练框架

事先在 OBS 桶中，新建用于上传代码和模型存储的文件夹

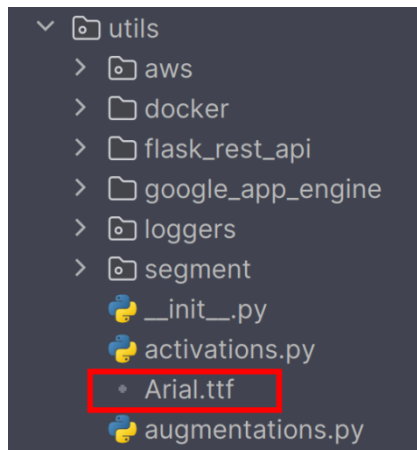


然后上传 PC 端的工程文件

桶列表 / test-dog-logistics / yolov5-7.0					
华北-北京四 桶内对象总数: 1426 存储总用量: 424.06 MB					
对象 版本					
上传 新建文件夹 下载 复制 更多 输入对象名前缀搜索					
对象名称	存储类别	大小	最后修改时间	操作	
.github	--	--	--	下载 分享 ...	
.idea	--	--	--	下载 分享 ...	
__pycache__	--	--	--	下载 分享 ...	
classify	--	--	--	下载 分享 ...	
data	--	--	--	下载 分享 ...	
dataset	--	--	--	下载 分享 ...	
models	--	--	--	下载 分享 ...	
runs	--	--	--	下载 分享 ...	
segment	--	--	--	下载 分享 ...	
utils	--	--	--	下载 分享 ...	
.dockerignore	标准存储	3.61 KB	2024/05/10 14:28:22 GMT...	下载 分享 ...	
.gitattributes	标准存储	75 bytes	2024/05/10 14:28:22 GMT...	下载 分享 ...	

其中我们需要做出如下修改：

1. 修改字体设置，防止在部署中因为网络原因报错



手动在浏览器中输入网址：<https://ultralytics.com/assets/Arial.ttf>，下载完成之后放在 utils 的目录下。

然后在 utils/general.py 文件中修改下面的代码，防止在训练的过程中因为下载字体原因导致卡住。

Error! Unknown document property name.

```
general.py × train.py detect.py data.yaml export.py
488 def check_font(font=FONT, progress=False):
489     # Download font to CONFIG_DIR if necessary
490     font = Path(font)
491     file = CONFIG_DIR / font.name
492     local_font_path = Path('yolov5-7.0/utils') / font.name
493     if local_font_path.exists():
494         return local_font_path
495     # if not font.exists() and not file.exists():
496     #     url = f'https://ultralytics.com/assets/{font.name}'
497     #     LOGGER.info(f'Downloading {url} to {file}...')
498     #     torch.hub.download_url_to_file(url, str(file), progress=progress)
499
```

```
1. def check_font(font=FONT, progress=False):
2.     # Download font to CONFIG_DIR if necessary
3.     font = Path(font)
4.     file = CONFIG_DIR / font.name
5.     local_font_path = Path('yolov5-7.0/utils') / font.name
6.     if local_font_path.exists():
7.         return local_font_path
8.     # if not font.exists() and not file.exists():
9.     #     url = f'https://ultralytics.com/assets/{font.name}'
10.    #     LOGGER.info(f'Downloading {url} to {file}...')
11.    #     torch.hub.download_url_to_file(url, str(file), progress=progress)
```

2. 设置模型的保存路径

```
general.py  train.py ×  detect.py  data.yaml  export.py
381         opt = vars(opt),
382         # 'git': GIT_INFO, # {remote, branch, commit} if a git repo
383         'date': datetime.now().isoformat()
384
385         # Save last, best and delete
386         torch.save(ckpt, last)
387         if best_fitness == fi:
388             torch.save(ckpt, best)
389         if opt.save_period > 0 and epoch % opt.save_period == 0:
390             torch.save(ckpt, w / f'epoch{epoch}.pt')
391
392         # 新增
393         model_save_path = opt.train_url
394         print(model_save_path)
395         save_name = os.path.join(model_save_path, 'model.pt')
396         torch.save(ckpt, save_name)
397
```

1. # 新增
2. model_save_path = opt.train_url
3. print(model_save_path)
4. save_name = os.path.join(model_save_path, 'model.pt')
5. torch.save(ckpt, save_name)

再设置一个参数

```
general.py  train.py ×  detect.py  data.yaml  export.py
441
442
443 def parse_opt(known=False):
444     parser = argparse.ArgumentParser()
445     parser.add_argument('--weights', type=str, default=ROOT / 'yolov5s.pt', help='initial weights path')
446     parser.add_argument('--cfg', type=str, default='', help='model.yaml path')
447     parser.add_argument('--data', type=str, default=ROOT / 'data/coco128.yaml', help='dataset.yaml path')
448     parser.add_argument('--train_url', type=str, default="weights", help='Train url for ModelArts')
449     parser.add_argument('--hyp', type=str, default=ROOT / 'data/hyps/hyp.scratch-low.yaml', help='hyperparameters path')
450     parser.add_argument('--epochs', type=int, default=100, help='total training epochs')
451     parser.add_argument('--batch-size', type=int, default=16, help='total batch size for all GPUs, -1 for autobatch')
452     parser.add_argument('--imgsz', '--img', '--img-size', type=int, default=640, help='train, val image size (pixels)')
453     parser.add_argument('--rect', action='store_true', help='rectangular training')
454     parser.add_argument('--resume', nargs='?', const=True, default=False, help='resume most recent training')
```

1. parser.add_argument('--train_url', type=str, default="weights", help='Train url for ModelArts')

3. Git 相关检测操作去除 在 modelarts 建立的环境中 git 相关操作可能因为网络原因导致脚本运行失败, git 相关操作在本实验中无用处, 可以去除, 先找到 train.py 中的导包部分, 再找红色圈出来的

Error! Unknown document property name.

```
general.py  train.py ×  detect.py  data.yaml  export.py
52         get_latest_run, increment_path, init_seeds, intersect_dicts, labels_to_class_weights,
53         labels_to_image_weights, methods, one_cycle, print_args, print_mutation, strip_optimizer,
54         yaml_save)
55 from utils.loggers import Loggers
56 from utils.loggers.comet.comet_utils import check_comet_resume
57 from utils.loss import ComputeLoss
58 from utils.metrics import fitness
59 from utils.plots import plot_evolve
60 from utils.torch_utils import (EarlyStopping, ModelEMA, de_parallel, select_device, smart_DDP, smart_optimizer,
61                               smart_resume, torch_distributed_zero_first)
62
63 LOCAL_RANK = int(os.getenv('LOCAL_RANK', -1)) # https://pytorch.org/docs/stable/elastic/run.html
64 RANK = int(os.getenv('RANK', -1))
65 WORLD_SIZE = int(os.getenv('WORLD_SIZE', 1))
66 # GIT_INFO = check_git_info()
67
68 os.environ["KMP_DUPLICATE_LIB_OK"]="TRUE"
69
```

1. `# GIT_INFO = check_git_info()`
- 2.
3. `os.environ["KMP_DUPLICATE_LIB_OK"]="TRUE"`

```
general.py  train.py ×  detect.py  data.yaml  export.py
376
377         'best_fitness': best_fitness,
378         'model': deepcopy(de_parallel(model)).half(),
379         'ema': deepcopy(ema.ema).half(),
380         'updates': ema.updates,
381         'optimizer': optimizer.state_dict(),
382         'opt': vars(opt),
383         # 'git': GIT_INFO, # {remote, branch, commit} if a git repo
384         'date': datetime.now().isoformat())
```

1. `# 'git': GIT_INFO, # {remote, branch, commit} if a git repo`

注意：确保上面的操作都已完成，才可进行下面的操作。

7.1.2. 文件上传 OBS

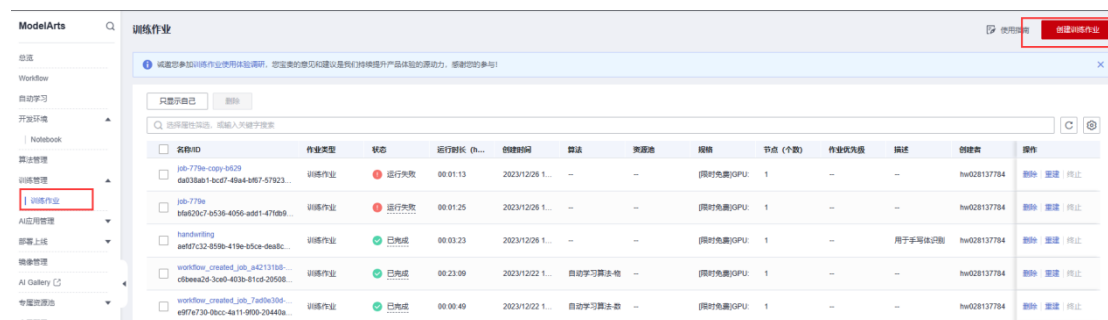
事先在 OBS 桶中，新建和上传代码、模型存储文件夹，文件夹和桶名称如下所示：



上传 ModelArts 端的工程文件。

7.1.3. 训练作业

进行训练之前，确保前面的所有步骤已经完成；在“训练管理”->“训练作业”，点击右上角的“创建训练作业”



新建一个训练作业，名称任意即可，同时按照下面配置，配置好路径和参数，在 modelarts 平台配置如下，训练作业名称：

* 名称	job-b956	✓
描述	智慧物流分拣	6/256

×

不支持选择跨区域（Region）的OBS桶。只能选择对象存储服务（OBS）桶下的文件夹。

对象存储服务（OBS）按照使用量收费。[了解计费详情](#)

确定

启动文件

×

不支持选择跨区域（Region）的OBS桶。请选择英文路径和英文命名文件。

test-dog-logistics / yolov5-7.0

请输入桶名称查询

新建桶 >>

桶名称

test-dog-logistics

新建文件夹

请输入名称查询

名称	最后修改时间	类型	大小
返回上一级			
tutorial.ipynb	2024/05/10 14:28:23 GM...	文件	53842 B
train.py	2024/05/10 14:28:23 GM...	文件	33913 B
val.py	2024/05/10 14:28:23 GM...	文件	20282 B
setup.cfg	2024/05/10 14:28:23 GM...	文件	1686 B
yolov5s.pt	2024/05/10 14:28:23 GM...	文件	1480843...
utils	--	文件夹	--
segment	--	文件夹	--
runs	--	文件夹	--

上一页 下一页

对象存储服务（OBS）按照使用量收费。 [了解计费详情](#)

取消

确定

* 创建方式

自定义算法

我的算法

我的订阅

?

* 启动方式

预置框架

自定义

* 代码目录

/test-dog-logistics/yolov5-7.0/

选择

?

* 启动文件

/test-dog-logistics/yolov5-7.0/train.py

选择

?

本地代码目录

/home/ma-user/modelarts/user-job-dir

工作目录

/home/ma-user/modelarts/user-job-dir

×

选择

输入参数：

Error! Unknown document property name.

输入 

▲

cfg

/test-dog-logistics/yolov5-7.0/models/yolov5s.yaml

数据集

数据存储位置

获取方式

☒ 超参

☐ 环境变量

cfg=/home/ma-user/modelarts/inputs/cfg_0

[+ 增加训练输入](#)

输出参数:

输出 

▲

train_url

/test-dog-logistics/output/

数据存储位置

获取方式

☒ 超参

☐ 环境变量

train_url=/home/ma-user/modelarts/outputs/train_url_0

预下载至本地目录 

☒ 不下载

☐ 下载

[+ 增加训练输出](#)

训练用到的超参数:

超参

img

=

640

batch

=

1

epoch

=

5

[+ 增加超参](#)

点击提交

信息确认

×

作业名	训练信息	价格
job-b956	算法	--
	预置镜像	PyTorch pytorch_1.8.0-cuda_10.2-py_3.7-ubuntu_18.04-x86_64
	代码目录	/test-dog-logistics/yolov5-7.0/
	启动文件	/test-dog-logistics/yolov5-7.0/train.py
	本地代码目录	/home/ma-user/modelarts/user-job-dir
	工作目录	/home/ma-user/modelarts/user-job-dir
	输入	cfg=/test-dog-logistics/yolov5-7.0/models/yolov5s.yaml
	输出	train_url=/test-dog-logistics/output/
	超参	img=640 batch=1 epoch=5

取消

确定

作业ID

ddc81e60-7e1d-4d67-b766-cb9878bd72af

作业状态

运行中

创建时间

2024/05/10 14:51:00 GMT+08:00

运行时长

00:00:54

描述

智慧物流分拣

预置镜像

PyTorch | pytorch_1.8.0-cuda_10.2-py_3.7-ubuntu_18.04-x86_64

算法名称

--

代码目录

/test-dog-logistics/yolov5-7.0/

启动文件

/test-dog-logistics/yolov5-7.0/train.py

运行用户ID

1102

本地代码目录

/home/ma-user/modelarts/user-job-dir

工作目录

/home/ma-user/modelarts/user-job-dir

事件

日志

资源占用情况

评估结果

标签

默认按照事件类型搜索、过滤

事件类型

事件信息

正常

[worker-0] 训练开始。

正常

训练作业开始运行。

正常

[Pod: modelarts-job-ddc81e60-7e1d-4d67-b766-cb9878bd72af-worker-0] Started: Started container

正常

[Pod: modelarts-job-ddc81e60-7e1d-4d67-b766-cb9878bd72af-worker-0] SuccessfulCreate: Create...

正常

[Pod: modelarts-job-ddc81e60-7e1d-4d67-b766-cb9878bd72af-worker-0] Pulled: Successfully pulled...

正常

[Pod: modelarts-job-ddc81e60-7e1d-4d67-b766-cb9878bd72af-worker-0] Pulling: pulling image "swr..."

正常

[Pod: modelarts-job-ddc81e60-7e1d-4d67-b766-cb9878bd72af-worker-0] Started: Started container

正常

[Pod: modelarts-job-ddc81e60-7e1d-4d67-b766-cb9878bd72af-worker-0] SuccessfulCreate: Create...

正常

[Pod: modelarts-job-ddc81e60-7e1d-4d67-b766-cb9878bd72af-worker-0] Pulled: Successfully pulled...

正常

[Pod: modelarts-job-ddc81e60-7e1d-4d67-b766-cb9878bd72af-worker-0] Pulling: pulling image "swr..."

等待完成

作业ID

85a0d70a-511b-4675-8ba9-d0b682835d56

作业状态

已完成

创建时间

2024/05/10 14:56:46 GMT+08:00

运行时长

00:01:46

描述

智慧物流分拣

预置镜像

PyTorch | pytorch_1.8.0-cuda_10.2-py_3.7-ubuntu_18.04-x86_64

算法名称

--

代码目录

/test-dog-logistics/yolov5-7.0/

启动文件

/test-dog-logistics/yolov5-7.0/train.py

运行用户ID

1102

本地代码目录

/home/ma-user/modelarts/user-job-dir

工作目录

/home/ma-user/modelarts/user-job-dir

系统日志

当前日志文件0MB;

请输入关键字查询

Service

time="2024-05-10T14:58:53+08:00" level=info msg="skip collecting mountstats, reason: MA_NFS_MOUNT_VOLUMES env is not found" file="collector.go:158" Command-bootstrap/run Component=ma-training-toolkit Platform=ModelArts-Service

time="2024-05-10T14:58:53+08:00" level=info msg="skip collecting ascend_log_parse_result, reason: not an ascend job, no need to collect" file="collector.go:158" Command-bootstrap/run Component=ma-training-toolkit Platform=ModelArts-Service

time="2024-05-10T14:58:53+08:00" level=info msg="skip collecting nfs_metrics, reason: MA_NFS_MOUNT_VOLUMES env is not found" file="collector.go:158" Command-bootstrap/run Component=ma-training-toolkit Platform=ModelArts-Service

time="2024-05-10T14:58:53+08:00" level=info msg="skip collecting ib_stats, reason: not an infiniband job" file="collector.go:158" Command-bootstrap/run Component=ma-training-toolkit Platform=ModelArts-Service

time="2024-05-10T14:58:53+08:00" level=info msg="skip collecting nvmlink_status, reason: empty results" file="collector.go:162" Command-bootstrap/run Component=ma-training-toolkit Platform=ModelArts-Service

time="2024-05-10T14:58:54+08:00" level=info msg="skip collecting node_lowlat, reason: no valid prom metrics in response" file="collector.go:158" Command-bootstrap/run Component=ma-training-toolkit Platform=ModelArts-Service

time="2024-05-10T14:58:54+08:00" level=info msg="skip collecting node_ssd_usage, reason: no valid prom metrics in response" file="collector.go:158" Command-bootstrap/run Component=ma-training-toolkit Platform=ModelArts-Service

time="2024-05-10T14:58:54+08:00" level=info msg="skip collecting fabric_manager_status, reason: no valid prometheus metrics in response" file="collector.go:158" Command-bootstrap/run Component=ma-training-toolkit Platform=ModelArts-Service

time="2024-05-10T14:58:56+08:00" level=info msg="post runtime info collection finished" file="run_train.go:1205" Command-bootstrap/run Component=ma-training-toolkit Platform=ModelArts-Service

time="2024-05-10T14:58:56+08:00" level=info msg="report event TrainingExit success" file="event.go:93" Command-bootstrap/run Component=ma-training-toolkit Platform=ModelArts-Service

time="2024-05-10T14:58:56+08:00" level=info msg="bootstrap is exiting with exit code 0" file="bootstrap.go:306" Command-bootstrap/run Component=ma-training-toolkit Platform=ModelArts-Service

time="2024-05-10T14:58:56+08:00" level=info msg="retcode 0 has been written to the retcode file /home/ma-user/modelarts/retcode" file="bootstrap.go:286" Command-bootstrap/run Component=ma-training-toolkit Platform=ModelArts-Service

运行成功，然后在模型的保存路径中找到模型文件

← → ↑

桶列表 / test-dog-logistics / output

华北-北京四

桶内对象总数: 1426

存储总用量: 424.06 MB

上传

新建文件夹

下载

复制

更多

对象名称

存储类别

model.pt

标准存储

7.1.4. 模型转换

在 obs 中将模型文件下载，然后上传项目文件中的 runs 目录下，然后运行：

```
1. python export.py --weights runs/model.pt --include onnx
```

```
runs
├── detect
├── train
├── model.onnx
└── model.pt

keras_model.summary()
if keras:
    keras_model.save(f, save_format='tf')
else:
    export_pb()
```

Terminal Local x + v

Successfully installed onnx-1.16.0 protobuf-5.26.1
YOLOv5 2024-4-10 Python-3.9.19 torch-2.2.2+cu121 CPU

Fusing layers...
YOLOv5s summary: 157 layers, 7023610 parameters, 0 gradients, 15.8 GFLOPs

PyTorch: starting from runs\model.pt with output shape (1, 25200, 10) (54.2 MB)

ONNX: starting export with onnx 1.16.0...
ONNX: export success 1.0s, saved as runs\model.onnx (27.2 MB)

Export complete (1.6s)
Results saved to D:\YSW\-\modelarts\yolov5-7.0\runs

Detect: python detect.py --weights runs\model.onnx
Validate: python val.py --weights runs\model.onnx
PyTorch Hub: model = torch.hub.load('ultralytics/yolov5', 'custom', 'runs\model.onnx')
Visualize: <https://netron.app>

在 runs 目录中成功生成了 model.onnx

将 ONNX 模型文件上传到开发板

```
(base) root@davinci-mini:/opt# ll
total 27888
drwxr-xr-x  2 root root    4096 Apr 12 17:10 ./
drwxr-xr-x 24 root root    4096 Apr  8 2022 ../
-rw-r--r--  1 root root 28545135 Apr 12 17:10 model.onnx
```

运行下面命令

1. atc --model=model.onnx --framework=5 --output=model --soc_version=Ascend310B4

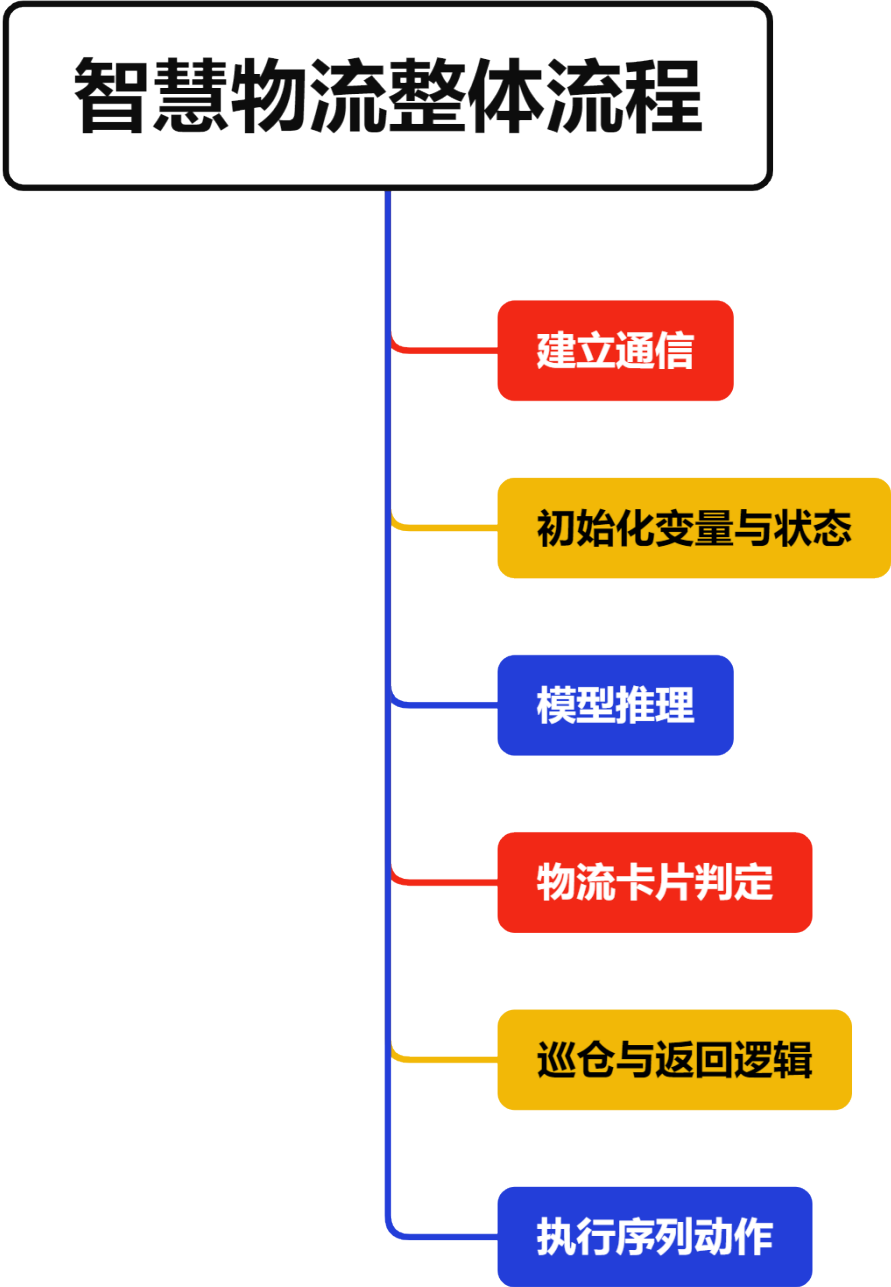
显示转换成功即可

```
(base) root@davinci-mini:/opt# atc --model=model.onnx --framework=5 --output=model --soc_version=Ascend310B4
ATC start working now, please wait for a moment.
...
ATC run success, welcome to the next use.
```

```
(base) root@davinci-mini:/opt# ll
total 42844
drwxr-xr-x  2 root root    4096 Apr 12 17:26 ./
drwxr-xr-x 24 root root    4096 Apr  8 2022 ../
-rw-r----- 1 root root    3022 Apr 12 17:26 fusion_result.json
-rw----- 1 root root 15307128 Apr 12 17:26 model.om
-rw-r--r--  1 root root 28545135 Apr 12 17:10 model.onnx
```

可以看到已经有 .om 模型文件输出。

第八章 逻辑实现



8.1. 导入必备的包

这部分导入了一系列必要的包和模块，为摄像头图像处理、深度学习推理以及机器狗控制提供支持。包含图片处理和多维数组计算，深度学习跟推理模块，工具以及通用包，摄像头图像的获取，模型前后的处理，还有机器狗控制相关包等。

```
1.  #!/usr/bin/env python
2.  # -*- coding: utf-8 -*-
3.
4.  import cv2 # 图片处理三方库，用于对图片进行前后处理
5.  import numpy as np # 用于对多维数组进行计算
6.  import torch # 深度学习运算框架，此处主要用来处理数据
7.  from mindx.sdk import Tensor # mxVision 中的 Tensor 数据结构
8.  from mindx.sdk import base # mxVision 推理接口
9.  import time
10. import logging
11. from typing import *
12. import sys
13. sys.path.append('camera/')
14. from HKcamera import getImage
15. from det_utils import get_labels_from_txt, letterbox, scale_coord  
s, nms, draw_bbox # 模型前后处理相关函数
16.
17. # 导入定义的机械狗相关的包
18. from threading_utils.ThreadTemplates import * # 线程的模板与基本轴指令的发送
19. from sendcommand import SendToCommand, heartbeat # 发送简单指令与心跳包
20. from socketnetwork import network_utils # 通信函数
21. from command.udp_command import * # 存放各种结构体与状态数据、指令
```

```
22. from speeds.sportspeed import *
```

```
# 运动精准
```

控制的基础版模型

8.2. 建立通信

简单控制第一步：建立通信链接，`socketnetwork/network_utils.py` 文件定义了一个函数 `setup_socket_and_address`，用于创建一个 UDP 套接字，并设置机器狗的 IP 地址和端口号作为目标地址。该通信链接是所有后续控制和数据传输的基础。不仅仅确保了指令传输的可靠性和及时性，还实现对机器狗的远程控制和监控以及为数据传输提供通道，代码如下：

```
1. import socket
2. from typing import *
3.
4. def setup_socket_and_address(dest_ip = '192.168.1.120', port=4389
5. ) -> Tuple[socket.socket, Tuple[str, int]]:
6.     # 创建UDP 套接字
7.     sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
8.
9.     # 设置目标地址
10.    target_address = (dest_ip, port)
11.    # print(target_address)
12.    return sfd, target_address
```

这段 Python 代码定义了一个函数 `setup_socket_and_address`，其目的是设置一个 UDP 套接字并返回该套接字及其目标地址。下面是对代码的逐行分析：

`import socket` - 导入 Python 的 `socket` 库，这个库提供了访问底层网络接口的机制。

`from typing import *` - 从 Python 的 `typing` 模块导入所有类型提示，用于在函数和方法

中添加类型注解，以提高代码的可读性和健壮性。

```
def setup_socket_and_address(dest_ip = '192.168.1.120', port=43893) ->
```

`Tuple[socket.socket, Tuple[str, int]]`: - 定义了一个名为 `setup_socket_and_address` 的函数，它接受两个参数：`dest_ip` 和 `port`，这两个参数分别指定了目标 IP 地址和端口号，默认值分别是 `'192.168.1.120'` 和 `43893`。函数的返回类型被注解为 `Tuple[socket.socket, Tuple[str, int]]`，意味着它将返回一个元组，其中包含一个 `socket` 对象和一个包含 IP 地址和端口号的元组。

`sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)` - 创建一个新的 UDP 套接字。`socket.AF_INET` 表示使用 IPv4 地址，`socket.SOCK_DGRAM` 表示使用 UDP 协议。

`target_address = (dest_ip, port)` - 将传入的 IP 地址和端口号组合成一个元组，这个元组将用作套接字的目标地址。

8.3. 发送心跳包

心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。这对于实时控制和数据传输至关重要。确保系统在运行过程中不会因为通信问题而出现故障，提高系统的整体可靠性。发送心跳包代码位于 `sendcommand/heartbeat.py`，代码如下：

```
1.  #!/usr/bin/env python
2.  # -*- coding: utf-8 -*-
3.
4.  import socket
5.  import struct
6.  import time
7.
```

```

8.     def send_udp_heartbeat(sfd, target_address, code=0x21040001, para
meters_size=0, type=0, heartbeat_interval=0.25) -> None:
9.         # 打包心跳指令数据
10.        heartbeat_command = struct.pack('<III', code, parameters_size
, type)
11.
12.        try:
13.            while True:
14.                # 记录发送前的时间
15.                start_time = time.time()
16.
17.                # 发送心跳包
18.                sfd.sendto(heartbeat_command, target_address)
19.
20.                # 发送后的延时
21.                time.sleep(max(0, heartbeat_interval - (time.time() -
start_time)))
22.
23.            except KeyboardInterrupt:
24.                print("Heartbeat sending stopped by user.")
25.        finally:
26.            # 关闭 socket
27.            sfd.close()

```

`network_utils.setup_socket_and_address()` 函数被调用，用于创建一个 UDP 套接字（或其他类型的网络通信接口）并设置机械狗的 IP 地址和端口号作为目标地址。心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。导入所需的模块：`socket` 用于网络通信，`struct` 用于数据打包，`time` 用于时间相关的操作。

定义了 `send_udp_heartbeat` 函数，它接收以下参数：

`sfd`: 一个 `socket` 对象，用于发送数据。

`target_address`: 目标地址的元组，包含 IP 地址和端口号。

`code`: 心跳指令的代码，默认为 `0x21040001`。

`parameters_size`: 参数大小，默认为 `0`。

`type`: 心跳包的类型，默认为 `0`。

`heartbeat_interval`: 心跳包发送间隔，默认为 `0.25` 秒。

使用 `struct.pack` 将心跳指令的数据打包成字节串。参数 `<III` 表示使用小端格式打包三个整数。

10-22. 在一个无限循环中，执行以下操作：

记录发送前的时间（第 15 行）。

发送心跳包到目标地址（第 18 行）。

计算发送后需要等待的时间，以确保心跳包的发送间隔符合 `heartbeat_interval`（第 21 行）。

23-24. 使用 `try-except` 结构来捕获 `KeyboardInterrupt` 异常，这通常发生在用户使用 `Ctrl+C` 中断程序时。如果捕获到这个异常，会打印一条消息表明心跳发送已被用户停止。

25-27. 使用 `finally` 块来确保无论是否发生异常，`socket` 都会在函数结束时关闭。

启动心跳线程，代码如下：

```
1.     # 建立通讯
2.     sfd, target_address = network_utils.setup_socket_and_address()
3.
4.     # 定义发送心跳包的线程执行体
5.     # 注意*args 必须以 tuple 形式传入，包括 sfd 和 target_address
```

```
6.     heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_udp_heartbeat, 0.25, None, sfd, target_address)
7.     heartbeat_thread.start()
```

`heartbeat_thread.start()` 启动了心跳线程，它将按照设定的时间间隔（这里是 0.25 秒）发送心跳包（`MyRepeatThread` 在后文中会解释）。

8.4. 创建指令头

创建指令头主要为了构建通信协议，提高代码的可读性和可维护性以及支持复杂的指令操作。指令头包含了关键信息，如指令码、参数大小和类型，是通信协议的核心组成部分。通过定义指令头，可以构建和解析复杂的通信协议。

```
1.     action_CommandHead = CommandHead()
```

创建了一个 `CommandHead` 对象，这个对象包含了指令的头部信息，如动作代码、序列号等。

```
1.     #!/usr/bin/env python
2.     # -*- coding: utf-8 -*-
3.
4.     import ctypes
5.     import struct
6.     import inspect
7.
8.     class CommandHead:
9.         def __init__(self, code=0, parameters_size=0, type_=0):
10.             self.code = code                                #
               指令码
11.             self.parameters_size = parameters_size          #
               指令值
12.             self.type_ = type_                                #
               指令类型
```

```

13.
14.     class Command:
15.         kDataSize = 256
16.
17.         def __init__(self, head=None, data=None):
18.             if head is None:
19.                 head = CommandHead()
20.             if data is None:
21.                 data = [0] * self.kDataSize
22.             self.head = head
23.             self.data = data
24.     class JointStateReceived(CommandHead):
25.         def __init__(self, data):
26.             """
27.
28.             1.关节信息的头部信息为 4 字节命令字,4 字节长度,4 字节类型,后续为
正文数据---(后续的所有都是类似)
29.
30.             2. 1 字节(b),2 字节无符号短整型(H),4 字节的无符号整型(I),接着是
12 个 8 字节的双精度浮点数(12d)
31.
32.             3.code,parameters_size,type_的是继承于 CommandHead 头的,这样
写是为了方便呈现出,数据都是基于 CommandHead 发送和接收的
33.             这样可以做到避免所有的复杂数据全部发在相同名字的 Command 中,而
是重新定义一个基于 CommandHead 的结构体
34.
35.             4.“self.code, ” 逗号是为了去除掉由 struct.unpack 操作产生
的元组问题,可以去去掉逗号直接复制
36.
37.             """

```

```

38.         self.data = data          # 关节的状态数据, 全部接受, 再由
code 来决定分配给谁

39.         CommandHead.code, = struct.unpack('<I', self.data[:4])

40.

41.

42.     class JointAngle(CommandHead):

43.         def __init__(self, joint_state_received):

44.             # 特定于关节角度的数据解析

45.             *self.joint_angles, = struct.unpack('<12d', joint_state_r
eceived.data[12:])

46.

47.     class JointSpeed(CommandHead):

48.         def __init__(self, joint_state_received):

49.             # 特定于关节速度的数据解析

50.             *self.joint_speeds, = struct.unpack('<12d', joint_state_r
eceived.data[12:])

51.

52.     class RobotState(CommandHead):

53.         def __init__(self, data):

54.

55.             CommandHead.code, = struct.unpack('<I', data[:4])

56.             CommandHead.parameters_size, = struct.unpack('<I', data[4
:8])

57.             CommandHead.type_, = struct.unpack('<I', data[8:12])

58.             self.robot_basic_state, = struct.unpack('<I', data[12:16]
)
                # 机器人基本运动状态

59.             self.robot_gait_state, = struct.unpack('<I', data[16:20])
                # 机器人步态信息

60.             self.robot_motion_state, = struct.unpack('<i', data[176:1
80])
                # 机器人动作状态

```

```

61.         self.distance_ahead, = struct.unpack('<d', data[-16:-
8]))
           # 雷达前方的距离
62.         self.rear_distance, = struct.unpack('<d', data[-8:])
63.
64.
65.         # self.rpy = struct.unpack('<3d', data[20:44])
           # IMU 角度
66.         # self.rpy_vel = struct.unpack('<3d', data[44:68])
           # IMU 角速度
67.         # self.xyz_acc = struct.unpack('<3d', data[68:92])
           # IMU 加速度
68.         # self.pos_world = struct.unpack('<3d', data[92:116])
           # 机器人在世界坐标系中的位置
69.         # self.vel_world = struct.unpack('<3d', data[116:140])
           # 机器人在世界坐标系中的速度
70.         # self.vel_body = struct.unpack('<3d', data[140:164])
           # 机器人在体坐标系中的速度
71.         # self.touch_down_and_stair_trot = struct.unpack('<I', da
ta[164:168]))
           # 此功能暂时未激活。此数据仅用于占位
72.         # self.is_charging = struct.unpack('<b', data[168:169])
           # 暂时未开放
73.         # self.error_state = struct.unpack('<I', data[169:173])
           # 暂时未开放
74.

```

类的继承结构

这些类都继承自 **CommandHead**，它们共享一些基本属性和方法。**CommandHead** 定义了基本信息，而他的子类定义了用于解析命令头部的方法和属性。

JointStateReceived 类

作用：这个类用于接收关节状态的原始数据，并且解析出头部信息。

解析过程：使用 `struct.unpack` 从数据的前 4 个字节解析出命令字（code）。这里使用了小端格式（<）和无符号整型（I）。

设计意图：通过继承 `CommandHead`，`JointStateReceived` 可以利用基类中定义的方法来处理命令头信息。

`JointAngle` 类

作用：专门用于从关节状态数据中解析出关节的角度。

解析过程：构造函数接收一个 `JointStateReceived` 实例，然后从实例的数据中（跳过头部 12 个字节的头部信息）解析出 12 个双精度浮点数，这些数表示关节的角度。

数据结构：关节角度数据紧跟在命令头之后，每个角度占用 8 个字节。

`JointSpeed` 类

作用：与 `JointAngle` 类似，但用于解析关节的速度。

解析过程：同样接收一个 `JointStateReceived` 实例，并从数据中解析出 12 个双精度浮点数表示关节速度。

`RobotState` 类

作用：解析机器人的整体状态，包括运动状态、步态信息、动作状态、雷达距离等。

解析过程：

首先解析命令头信息，包括命令码、参数大小和类型。

然后解析机器人的基本运动状态、步态信息、动作状态等，每个状态占用 4 个字节。

数据解析的细节

小端格式： `struct.unpack` 中的 < 表示小端格式，这是计算机存储多字节数据时的一种字节顺序，其中最低有效字节存储在地址的最低位。

数据类型：使用不同的格式字符来指定数据类型，例如 I 表示无符号整型（4 字节），d 表示双精度浮点数（8 字节）。

星号(*)：在 `struct.unpack` 中使用星号是为了接收多个返回值，并将它们解包到多个变

量中。

设计哲学

模块化：每个类负责解析特定部分的数据，这有助于代码的组织和维护。

扩展性：通过继承 **CommandHead**，新的状态解析类可以轻松地添加，同时复用头部解析逻辑。

灵活性：通过注释掉一些解析字段，代码提供了在未来添加新功能的可能性。

使用场景

这些类用于一个机器狗的控制系统中，用于实时监控和解析机器人的状态信息，以便于进行决策和控制。

8.5. 图像处理函数

包含了多个用于图像处理和目标检测的函数。共同组成了一套完整的图像处理和目标检测工具，涵盖了图像预处理、坐标变换、非极大值抑制以及检测结果的可视化等各个方面。从图像预处理到最终的检测结果可视化，提供了一套完整的流程。

1.

导入 time 模块，提供各种与时间相关的函数，例如时间戳、休眠等。

2. `import time`

3.

4. *# 导入 OpenCV 库，一个开源的计算机视觉和机器学习软件库，用于图像和视频处理。*

5. `import cv2`

6.

7. *# 导入 NumPy 库，一个用于科学计算的 Python 库，提供多维数组对象、派生对象（如掩码数组、矩阵）以及用于快速数组操作的函数。*

8. `import numpy as np`

9.

10. *# 导入 PyTorch 库，一个开源的机器学习库，基于 Torch 库，用于深度学习，提供自动微分、GPU 加速等功能。*

```

11. import torch
12.
13. # 导入 torchvision 库，一个PyTorch 的扩展包，包含处理图像和视频的实用工具
    和预训练模型。
14. import torchvision
15.
16. # 定义 letterbox 函数，参数 new_shape 指定目标尺寸，默认为 (640, 640)，
    color 指定边框颜色，默认为灰色 (114, 114, 114)。
17. # auto 参数决定是否使用最小矩形，scaleFill 参数决定是否拉伸图像填充新尺寸，
    scaleup 参数决定是否放大图像。
18. def letterbox(img, new_shape=(640, 640), color=(114, 114, 114), auto=False,
    scaleFill=False, scaleup=True):
19.     # 获取当前图像的尺寸 [height, width]
20.     shape = img.shape[:2]
21.
22.     # 如果 new_shape 是一个整数，则将其转换为一个元组
    (new_shape, new_shape)
23.     if isinstance(new_shape, int):
24.         new_shape = (new_shape, new_shape)
25.
26.     # 计算新旧尺寸的比例，取宽度和高度比例的最小值作为缩放比例
27.     r = min(new_shape[0] / shape[0], new_shape[1] / shape[1])
28.
29.     # 如果 scaleup 为 False，则限制 r 的值不超过 1，即不放大图像
30.     if not scaleup:
31.         r = min(r, 1.0)
32.
33.     # 计算缩放后的图像尺寸
34.     new_unpad = int(round(shape[1] * r)), int(round(shape[0] * r))
35.

```

```

36.         # 计算宽度和高度的填充量
37.         dw, dh = new_shape[1] - new_unpad[0], new_shape[0] -
            new_unpad[1] # wh padding
38.
39.         # 如果 auto 为 True, 则使用最小矩形, 填充量应为 64 的倍数
40.         if auto:
41.             dw, dh = np.mod(dw, 64), np.mod(dh, 64) # wh padding
42.
43.         # 如果 scaleFill 为 True, 则不进行缩放, 直接使用新尺寸填充
44.         elif scaleFill:
45.             dw, dh = 0.0, 0.0
46.             new_unpad = (new_shape[1], new_shape[0]) # 使用新尺寸
47.             ratio = new_shape[1] / shape[1], new_shape[0] / shape[0]
            # 重新计算宽高比例
48.
49.         # 将填充量分为上下和左右两部分
50.         dw /= 2
51.         dh /= 2
52.
53.         # 如果需要调整图像尺寸, 则进行缩放
54.         if shape[:,-1] != new_unpad:
55.             img = cv2.resize(img, new_unpad, interpolation=cv2.INTER_
                LINEAR)
56.
57.         # 计算上、下、左、右边界的填充量, 并进行四舍五入
58.         top, bottom = int(round(dh - 0.1)), int(round(dh + 0.1))
59.         left, right = int(round(dw - 0.1)), int(round(dw + 0.1))
60.
61.         # 使用 cv2.copyMakeBorder 函数添加边框, 颜色为 color 指定的颜色

```

```

62.     img = cv2.copyMakeBorder(img, top, bottom, left, right, cv2.B
    ORDER_CONSTANT, value=color)
63.
64.     # 返回调整尺寸并添加边框后的图像，缩放比例，以及左右和上下的填充量
65.     return img, ratio, (dw, dh)
66.
67. # 定义 non_max_suppression 函数，用于在推理结果上执行非极大值抑制
    (NMS)，以排除重叠的检测
68. def non_max_suppression(
69.     prediction, # 预测结果，是模型输出的张量
70.     conf_thres=0.25, # 置信度阈值，默认为0.25
71.     iou_thres=0.45, # 交并比 (IoU) 阈值，默认为0.45
72.     classes=None, # 指定要检测的类别列表，如果为None，则检测所有类别
73.     agnostic=False, # 是否进行类别不可知的NMS
74.     multi_label=False, # 是否允许单个实例有多个标签
75.     labels=(), # 可选的标签列表，用于某些模型的自动标签注入
76.     max_det=300, # 每张图像上最多检测的目标数量
77.     nm=0, # 掩码数量，当前未使用
78. ):
79.     """在推理结果上执行非极大值抑制（NMS），以排除重叠的检测框
80.     返回值：
81.         每张图像的检测结果列表，格式为 (n,6) 张量 [xyxy, conf, cls]
82.         """
83.
84.     # 如果预测结果是一个列表或元组（例如在使用 YOLOv5 模型的验证模式时），
    则只选择推理输出
85.     if isinstance(prediction, (list, tuple)):
86.         prediction = prediction[0]
87.
88.     # 获取预测结果所在的设备（CPU 或 GPU）

```

```

89.     device = prediction.device
90.
91.     # 检查是否在使用Apple MPS (Metal Performance Shaders), 目前MPS
    对NMS 的支持不完整
92.     mps = 'mps' in device.type
93.     # 如果正在使用MPS, 则在执行NMS 前将张量转换到CPU
94.     if mps:
95.         prediction = prediction.cpu()
96.
97.     # 获取批量中的图像数量
98.     bs = prediction.shape[0]
99.
100.    # 计算类别数量, 减去5 是为了去掉模型输出中的5 个固定元素, nm 是掩码
    数量, 当前未使用
101.    nc = prediction.shape[2] - nm - 5
102.
103.    # 根据置信度阈值过滤预测结果, xc 为置信度大于conf_thres 的候选预测
    结果
104.    xc = prediction[..., 4] > conf_thres
105.
106.    # 下面的代码将继续实现NMS 算法, 筛选出最佳的检测结果
107.    # 检查置信度阈值是否在0 到1 之间
108.    assert 0 <= conf_thres <= 1, f'Invalid Confidence threshold {conf
    _thres}, valid values are between 0.0 and 1.0'
109.    # 检查交并比阈值是否在0 到1 之间
110.    assert 0 <= iou_thres <= 1, f'Invalid IoU {iou_thres}, valid valu
    es are between 0.0 and 1.0'
111.
112.    # 设置
113.    # min_wh = 2  # 检测框的最小宽度和高度 (像素)

```

```

114. max_wh = 7680 # 检测框的最大宽度和高度（像素）
115. max_nms = 30000 # 传入 torchvision.ops.nms() 的最大检测框数量
116. time_limit = 0.5 + 0.05 * bs # 执行 NMS 后退出的时间限制（秒）
117. # 如果类别数大于1，则允许每个检测框有多个标签
118. multi_label &= nc > 1
119.
120. # 记录当前时间，用于监控 NMS 的执行时间
121. t = time.time()
122. # 掩码的起始索引，5 是因为模型输出的前5个值不是类别概率
123. mi = 5 + nc
124. # 初始化输出列表，用于存储每张图像的检测结果
125. output = [torch.zeros((0, 6 + nm), device=prediction.device)] * b
    s
126.
127. # 遍历批量中的每张图像
128. for xi, x in enumerate(prediction):
129.     # 如果没有检测框剩余，则处理下一张图像
130.     if not x.shape[0]:
131.         continue
132.
133.     # 如果启用自动标注，则将先验标签添加到预测结果中
134.     if labels and len(labels[xi]):
135.         lb = labels[xi]
136.         # 创建一个零张量用于存放自动标注的标签信息
137.         v = torch.zeros((len(lb), nc + nm + 5), device=x.device)
138.         # 将标签的边界框信息复制到新张量
139.         v[:, :4] = lb[:, 1:5]
140.         # 设置置信度为1.0
141.         v[:, 4] = 1.0
142.         # 设置类别标签

```

```

143.         v[range(len(lb)), lb[:, 0].long() + 5] = 1.0
144.         # 将自动标注的标签信息添加到预测结果中
145.         x = torch.cat((x, v), 0)
146.
147.         # 计算置信度，将对象置信度与类别置信度相乘
148.         x[:, 5:] *= x[:, 4:5]
149.
150.         # 将中心点坐标和宽高转换为左上角和右下角坐标
151.         box = xywh2xyxy(x[:, :4])
152.         # 获取掩码信息，如果没有掩码，则为零列
153.         mask = x[:, mi:]
154.
155.         # 如果允许多标签，则为每个类别分配一个标签，否则只保留最佳类别
156.         if multi_label:
157.             i, j = (x[:, 5:mi] > conf_thres).nonzero(as_tuple=False).
                T
158.             x = torch.cat((box[i], x[i, 5 + j, None], j[:, None].float(),
                t(), mask[i]), 1)
159.         else:
160.             conf, j = x[:, 5:mi].max(1, keepdim=True)
161.             x = torch.cat((box, conf, j.float(), mask), 1)[conf.view(
                -1) > conf_thres]
162.
163.         # 如果指定了要检测的类别，则过滤其他类别的检测框
164.         if classes is not None:
165.             x = x[(x[:, 5:6] == torch.tensor(classes, device=x.device))
                .any(1)]
166.
167.         # 检查检测框数量
168.         n = x.shape[0]

```

```

169.     if not n: # 如果没有检测框，则跳过
170.         continue
171.     elif n > max_nms: # 如果检测框数量超过限制，则按置信度排序后取前
        max_nms 个
172.         x = x[x[:, 4].argsort(descending=True)[:max_nms]]
173.     else:
174.         # 如果检测框数量没有超过限制，则按置信度排序
175.         x = x[x[:, 4].argsort(descending=True)]
176.
177.     # 执行批量 NMS
178.     c = x[:, 5:6] * (0 if agnostic else max_wh) # 类别偏移
179.     boxes, scores = x[:, :4] + c, x[:, 4] # 检测框（偏移类别），分
        数
180.     # 执行 NMS，返回保留的检测框索引
181.     i = torchvision.ops.nms(boxes, scores, iou_thres)
182.
183.     # 如果检测到的物体数量超过最大检测数量，则只保留前 max_det 个
184.     if i.shape[0] > max_det:
185.         i = i[:max_det]
186.
187.     # 将当前图像的检测结果添加到输出列表中
188.     output[xi] = x[i]
189.     # 如果在使用 Apple MPS，则将结果移回原始设备
190.     if mps:
191.         output[xi] = output[xi].to(device)
192.
193.     # 如果 NMS 执行时间超过时间限制，则打印警告并退出循环
194.     if (time.time() - t) > time_limit:
195.         print(f'WARNING ⚠️ NMS time limit {time_limit:.3f}s excee
            ded')

```



```

196.         break
197.
198. # 返回所有图像的检测结果
199. return output
200.
201. # 定义 xywh2xyxy 函数，用于将 nx4 边界框从 [x, y, w, h] 格式转换为
    [x1, y1, x2, y2] 格式
202. # 其中 xy1 表示左上角坐标，xy2 表示右下角坐标
203. def xywh2xyxy(x):
204.     # 如果输入是 torch.Tensor 类型，则克隆数据；如果是 numpy 数组，则
        复制数据
205.     y = x.clone() if isinstance(x, torch.Tensor) else np.copy(x)
206.     # 计算左上角 x 坐标
207.     y[:, 0] = x[:, 0] - x[:, 2] / 2
208.     # 计算左上角 y 坐标
209.     y[:, 1] = x[:, 1] - x[:, 3] / 2
210.     # 计算右下角 x 坐标
211.     y[:, 2] = x[:, 0] + x[:, 2] / 2
212.     # 计算右下角 y 坐标
213.     y[:, 3] = x[:, 1] + x[:, 3] / 2
214.     return y
215.
216. # 定义 get_labels_from_txt 函数，用于从文本文件中读取标签并创建字典
217. def get_labels_from_txt(path):
218.     labels_dict = dict() # 初始化一个空字典来存储标签
219.     with open(path) as f: # 打开文件
220.         for cat_id, label in enumerate(f.readlines()): # 遍历文件
            中的每一行
221.             labels_dict[cat_id] = label.strip() # 将读取的标签去除
                两端空白后存入字典

```

```

222.     return labels_dict # 返回标签字典
223.
224. # 定义 scale_coords 函数, 用于调整坐标点以适应不同尺寸的图像
225. def scale_coords(img1_shape, coords, img0_shape, ratio_pad=None):
226.     # 根据 img1_shape 和 img0_shape 调整坐标
227.     if ratio_pad is None: # 如果没有提供 ratio_pad, 则从
        img0_shape 计算
228.         gain = min(img1_shape[0] / img0_shape[0], img1_shape[1] /
            img0_shape[1]) # 计算缩放比例
229.         pad = (img1_shape[1] -
            img0_shape[1] * gain) / 2, (img1_shape[0] -
            img0_shape[0] * gain) / 2 # 计算填充量
230.     else: # 如果提供了 ratio_pad, 则使用提供的值
231.         gain = ratio_pad[0][0]
232.         pad = ratio_pad[1]
233.
234.     # 应用 x 方向的填充
235.     coords[:, [0, 2]] -= pad[0]
236.     # 应用 y 方向的填充
237.     coords[:, [1, 3]] -= pad[1]
238.     # 应用缩放比例
239.     coords[:, :4] /= gain
240.     # 裁剪坐标点, 确保它们在有效范围内
241.     clip_coords(coords, img0_shape)
242.     return coords # 返回调整后的坐标点
243.
244. # 定义 clip_coords 函数, 用于裁剪边界框坐标, 确保它们在图像尺寸范围内
245. def clip_coords(boxes, shape):
246.     # 如果 boxes 是 torch.Tensor 类型, 则逐个快速裁剪
247.     if isinstance(boxes, torch.Tensor):

```

```

248.         boxes[:, 0].clamp_(0, shape[1]) # 裁剪 x1 坐标, 确保在 0 到
           图像宽度之间
249.         boxes[:, 1].clamp_(0, shape[0]) # 裁剪 y1 坐标, 确保在 0 到
           图像高度之间
250.         boxes[:, 2].clamp_(0, shape[1]) # 裁剪 x2 坐标, 确保在 0 到
           图像宽度之间
251.         boxes[:, 3].clamp_(0, shape[0]) # 裁剪 y2 坐标, 确保在 0 到
           图像高度之间
252.     # 如果 boxes 是 numpy 数组, 则整体快速裁剪
253.     else:
254.         boxes[:, [0, 2]] = boxes[:, [0, 2]].clip(0, shape[1]) #
           裁剪 x1 和 x2
255.         boxes[:, [1, 3]] = boxes[:, [1, 3]].clip(0, shape[0]) #
           裁剪 y1 和 y2
256.
257. # 定义 nms 函数, 作为非极大值抑制 (NMS) 的封装, 调用
           non_max_suppression 函数
258. def nms(box_out, conf_thres=0.4, iou_thres=0.5):
259.     try:
260.         # 尝试使用 multi_label=True 的 NMS
261.         boxout = non_max_suppression(box_out, conf_thres=conf_thr
           es, iou_thres=iou_thres, multi_label=True)
262.     except:
263.         # 如果出错, 则使用默认的 NMS
264.         boxout = non_max_suppression(box_out, conf_thres=conf_thr
           es, iou_thres=iou_thres)
265.     return boxout # 返回 NMS 处理后的结果
266.
267. # 定义 draw_bbox 函数, 在图像上绘制边界框和标签
268. def draw_bbox(bbox, img0, color, wt, names):

```

```

269.     det_result_str = '' # 初始化检测结果字符串
270.     for idx, class_id in enumerate(bbox[:, 5]): # 遍历边界框和类别
        ID
271.         if float(bbox[idx][4]) < float(0.05): # 如果置信度小于
            0.05, 则跳过
272.             continue
273.         print(str(idx) + ' ' + names[int(class_id)]) # 打印边界框
            索引和类别名称
274.         # 在图像上绘制矩形边界框
275.         img0 = cv2.rectangle(img0, (int(bbox[idx][0]), int(bbox[i
            dx][1])), (int(bbox[idx][2]), int(bbox[idx][3])), color, wt)
276.         # 在边界框左上角绘制类别名称
277.         img0 = cv2.putText(img0, str(idx) + ' ' + names[int(class
            _id)], (int(bbox[idx][0]), int(bbox[idx][1] + 16)), cv2.FONT_HERSHEY_
            _SIMPLEX, 0.5, (0, 0, 255), 1)
278.         # 在边界框下方绘制置信度
279.         img0 = cv2.putText(img0, '{:.2f}'.format(bbox[idx][4]), (
            int(bbox[idx][0]), int(bbox[idx][1] + 32)), cv2.FONT_HERSHEY_SIMPLEX,
            0.5, (0, 0, 255), 1)
280.         # 将检测结果添加到字符串
281.         det_result_str += '{ } { } { } { } { }\n'.format(names[bbox
            [idx][5]], str(bbox[idx][4]), bbox[idx][0], bbox[idx][1], bbox[idx][
            2], bbox[idx][3])
282.     return img0 # 返回绘制了边界框和标签的图像

```

letterbox: 这个函数用于将图像调整到指定的新尺寸，同时保持长宽比，通过添加边框来填充图像。它还计算了缩放比例和填充尺寸。

non_max_suppression: 这个函数实现了非极大值抑制（NMS），用于从预测结果中去除重叠的检测框。它接受多个参数，包括置信度阈值、IoU 阈值等，并返回经过 NMS 处理的检测结果。

xywh2xyxy: 将坐标格式从(x, y, w, h)转换为(x1, y1, x2, y2), 即从中心点和宽高转换为左上角和右下角的坐标。

get_labels_from_txt: 从文本文件中读取类别标签, 并将它们存储在字典中。

scale_coords: 将坐标从一个图像尺寸调整到另一个图像尺寸, 考虑了缩放和平移。

clip_coords: 将坐标限制在给定图像尺寸范围内, 确保检测框不会超出图像边界。

nms: 一个包装函数, 用于调用 **non_max_suppression** 函数并返回 NMS 处理后的检测框。

draw_bbox: 在图像上绘制检测框, 并在框旁边显示类别名称和置信度。

8.6. 速度函数

这段代码提供了一组用于控制设备运动的函数, 包括直线行驶、左右平移和左右旋转。每个运动函数都接受一些参数, 如速度档位和运动的距离或时间, 并计算出所需的时间和速度。代码中使用了条件逻辑来根据不同的情况选择不同的计算公式。旋转函数中使用了栈跟踪来确定调用上下文, 代码是为了适应不同的使用场景而设计的。

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. from command.udp_command import *                # 存放各种结构
   体与状态数据、指令
5.
6. def go_straight(long, speedgear=3, times = None):
7.     # 档位速度只是参考值, 具体速度按照实际来判断, 默认速度是3档
8.     speedgear_ditc = {
9.         1:7000,
```

```

10.         2:7500,
11.         3:8000,
12.         4:8700,
13.         5:9000,
14.         6:10500,
15.     }
16.
17.     val = speedgear_ditc[speedgear]
18.     # 如果传入的是时间, 计算出已经走过的距离, long 传入 9999 是为在特殊情况才
能会有下面的情况
19.     if times != None and long == 9999:
20.         if long < 0:
21.             val = -val
22.             # speed_per_second_meter = (-1.086e-08) * val ** 2 -
0.0003272 * val - 1.698
23.             speed_per_second_meter = (-1.5059e-08) * val ** 2 -
(4.1944e-04) * val - 2.1804
24.             long = abs(times * speed_per_second_meter)
25.         else:
26.             speed_per_second_meter = (-7.237e-
09) * val ** 2 + 0.0002933 * val - 1.643
27.             long = abs(times * speed_per_second_meter)
28.         return long
29.

```

```

30.     else:
31.         if long < 0:
32.             val = -val
33.             # speed_per_second_meter = (-1.086e-08) * val ** 2 -
              0.0003272 * val - 1.698
34.             speed_per_second_meter = (-1.5059e-08) * val ** 2 -
              (4.1944e-04) * val - 2.1804
35.             times = abs(long / speed_per_second_meter)
36.         else:
37.             speed_per_second_meter = (-7.237e-
              09) * val ** 2 + 0.0002933 * val - 1.643
38.             times = long / speed_per_second_meter
39.         return [times, val]
40.
41.
42. def translate_left_and_right(long, speedgear=3):
43.     # 档位速度只是参考值，具体速度按照实际来判断，默认速度是3档
44.     speedgear_ditc = {
45.         1: 14000,
46.         2: 18000,
47.         3: 21000,
48.         4: 24000,
49.         5: 27000,
50.         6: 34000,

```

```

51.     }

52.     val = speedgear_ditc[speedgear]

53.     if long < 0:

54.         val = -val

55.         speed_per_second_meter = -(1.6e-05) * val - 0.2256

56.         times = abs(long / speed_per_second_meter)

57.     else:

58.         speed_per_second_meter = (1.517e-05) * val - 0.1748

59.         times = long / speed_per_second_meter

60.     return [times, val]

61.

62.

63. def revolve_left_and_right(angle):

64.     def find_closest_output(input_value):

65.         # 判断角度是否超过了360度, 超过了则需要则算掉

66.         if input_value > 360:

67.             input_value = input_value / (input_value % 360)

68.

69.         # 输出到输入的映射字典

70.         output_to_input_map = {

71.             0: 0,

72.             15: 0.1,

```


73.	30: 0.2,
74.	45: 0.53,
75.	60: 0.6,
76.	75: 0.9,
77.	90: 1.3,
78.	105: 1.5,
79.	120: 1.9,
80.	135: 2.2,
81.	150: 2.3,
82.	165: 2.5,
83.	167: 2.8,
84.	185: 2.9,
85.	195: 3.1,
86.	210: 3.4,
87.	225: 3.7,
88.	240: 4,
89.	255: 4.3,
90.	270: 4.5,
91.	285: 4.7,
92.	300: 5,
93.	315: 5.3,
94.	330: 5.6,

```

95.         345: 5.8,
96.         360: 5.9,
97.     }
98.
99.
100.
101.         # 获取所有输出键并将其转换为列表
102.         keys = list(output_to_input_map.keys())
103.         # 寻找与输入值最接近的输出键
104.         closest_key = min(keys, key=lambda x: abs(x -
            input_value))
105.         # 返回与最接近键关联的输入值
106.         return output_to_input_map[closest_key]
107.
108.     caller_function = StackTraceParser()
109.     # print(caller_function.get_formatted_stack_info())
110.
111.     # 根据栈的追踪, 如果旋转的调用执行不同的旋转方式
112.     if caller_function.get_stack_info()[0]['file'].split('/')[1] == 'hand_distance_main.py':
113.         if 8 < abs(angle) < 20:
114.             if angle < 0:
115.                 val = -10000

```

```
116.         else:
117.             val = 10000
118.             times = 0.1
119.         elif 20 <= abs(angle):
120.             if angle < 0:
121.                 val = -10000
122.             else:
123.                 val = 10000
124.                 times = 0.2
125.         else:
126.             times, val = 0, 0
127.
128.     else:
129.         times = find_closest_output(abs(angle)) # 获取时间与角度的
            关系
130.         if angle < 0:
131.             val = -10000
132.         else:
133.             val = 10000
134.
135.     return [times, val]
136.
137.
```

```
go_straight(long, speedgear=3, times=None):
```

这个函数用于控制设备直线行驶。

long: 表示要行驶的距离，单位是米。

speedgear: 表示设备的档位，影响速度，其默认值为3。

times: 表示行驶时间，单位是秒。

函数首先定义了一个字典speedgear_ditc，它将档位映射到一个速度值，如果times不为None且long等于9999，函数会根据时间和速度计算行驶的距离。

如果long不是9999，函数会根据距离和速度计算所需的时间。

最终，函数返回行驶所需的时间（或距离）和速度值。

```
translate_left_and_right(long, speedgear=3):
```

这个函数用于控制设备左右平移。

参数和go_straight函数类似，但速度值是不同的字典speedgear_ditc。

根据long的正负，选择相应的速度计算公式来计算时间。

返回值是平移所需的时间和速度值。

```
revolve_left_and_right(angle):
```

这个函数用于控制设备左右旋转。

angle: 表示要旋转的角度，单位是度。

函数内部定义了一个辅助函数find_closest_output，用于找到最接近输入角度的输出值。

函数使用了一个映射字典output_to_input_map，将角度映射到一个输出值。

还有一个未定义的StackTraceParser类和它的get_formatted_stack_info方法，用于确定是哪个文件调用了旋转函数，以便执行不同的旋转逻辑。

根据调用的文件名和角度的大小，函数会设置不同的时间和速度值。

最终，函数返回旋转所需的时间和速度值。

8.7. 模型初始化

定义一个初始化模型的函数。主要作用是初始化深度学习模型并加载相关的标签信息，以便进行模型推理和结果解释。以下是对文件的总体总结和作用的描述：

```

1.     def model_init(model_path, device_id, label_path) -> Tuple['base.
model', Dict[int, str]]:
2.         # 初始化资源和变量
3.         base.mx_init() # 初始化 mxVision 资源
4.         model = base.model(modelPath=model_path, deviceId=device_id)
        # 初始化 base.model 类
5.         labels_dict = get_labels_from_txt(label_path) # 得到类别信
息, 返回序号与类别对应的字典
6.         return model, labels_dict

```

函数定义:

`model_init` 函数接受三个参数: `model_path` (模型路径), `device_id` (设备 ID), 和 `label_path` (标签文件路径)。

函数返回一个元组, 包含一个 `base.model` 类的实例和一个字典。字典的键是整数, 值是字符串, 表示类别的序号和对应的名称。

初始化 `mxVision` 资源:

调用 `base.mx_init()` 函数来初始化 `mxVision` 库的资源。`mxVision` 是一个推理框架, 通常用于深度学习模型的部署和推理。

加载模型:

使用 `base.model(modelPath=model_path, deviceId=device_id)` 创建一个 `base.model` 类的实例。这个实例将用于加载和执行深度学习模型。

`modelPath` 参数指定了模型文件的路径, `deviceId` 指定了要使用的设备 ID (例如, CPU 或 GPU)。

加载标签信息:

调用 `get_labels_from_txt(label_path)` 函数来从文本文件中加载标签信息。这个函数应该返回一个字典, 其中包含类别的序号 (作为键) 和对应的类别名称 (作为值)。

返回值:

函数返回两个对象: 初始化的模型实例和标签字典。这样, 调用者就可以使用这些对象来进行模型推理和结果的解释。

8.8. 模型推理

这一部分主要作用是执行图像推理过程，并返回推理结果的列表。从图像的获取到最终结果的解析和返回，提供了一个端到端的解决方案。以下是对文件的总体总结和作用的描述

```
1.     def Image_inference(model, labels_dict) -> list:
2.         try:
3.             while True:
4.                 img = getImage()
5.                 frame = img.copy()
6.                 # 数据前处理
7.                 img, scale_ratio, pad_size = letterbox(frame, new_shape=[640, 640]) # 对图像进行缩放与填充，保持长宽比
8.                 # 用来存放推理结构，三个元素，[idx: 识别到的序列号（指的是一次性识别到4个东西，那么序列号就是0,1,2,3），class_id: 识别到的类型，pred_all[idx][4]: 识别率]
9.                 ill_sets = []
10.                img = img[:, :, :-
11.                1].transpose(2, 0, 1) # BGR to RGB, HWC to CHW
12.                img = np.expand_dims(img, 0).astype(np.float32) # 将形状转换为 channel first (1, 3, 640, 640)，即扩展第一维为 batchsize
13.                img = np.ascontiguousarray(img) / 255.0 # 转换为内存连续存储的数组
14.                img = Tensor(img) # 将numpy 转为Tensor 类
15.                # 模型推理，得到模型输出
16.                # model = base.model(modelPath=model_path, deviceId=DEVICE_ID) # 初始化 base.model 类
```

```

17.         output = model.infer([img])[0] # 执行推理。输入数据类型: List[base.Tensor], 返回模型推理输出的 List[base.Tensor]

18.

19.         # 后处理

20.         output.to_host() # 将 Tensor 数据转移到 Host 侧

21.         output = np.array(output) # 将数据转为 numpy array 类型

22.         boxout = nms(torch.tensor(output), conf_thres=0.4, iou_thres=0.5) # 利用非极大值抑制处理模型输出, conf_thres 为置信度阈值, iou_thres 为iou 阈值

23.         pred_all = boxout[0].numpy() # 转换为numpy 数组

24.         scale_coords([640, 640], pred_all[:, :4], frame.shape, ratio_pad=(scale_ratio, pad_size)) # 将推理结果缩放到原始图片大小

25.         # labels_dict = get_labels_from_txt('ceshi/demo_labels.txt') # 得到类别信息, 返回序号与类别对应的字典

26.         # img_dw = draw_bbox(pred_all, frame, (0, 255, 0), 2, labels_dict) # 画出检测框、类别、概率

27.

28.         for idx, class_id in enumerate(pred_all[:, 5]):

29.             logging.info(str(idx) + ' ' + labels_dict[int(class_id)])

30.             # 识别率保留两位小数

31.             ill_sets.append([idx, int(class_id), round(pred_all[idx][4], 2)])

32.

33.         if ill_sets == []:

34.             pass

35.         else:

36.             return ill_sets

37.

```

```
38.  
39.         except KeyboardInterrupt:  
40.             pass  
41.  
42.         finally:  
43.             pass
```

函数定义:

`Image_inference` 函数接受两个参数: `model` (深度学习模型的实例) 和 `labels_dict` (类别标签的字典)。

函数返回一个列表, 其中包含识别结果的详细信息。

无限循环获取图像:

使用 `getImage()` 函数获取图像, 并复制一份用于后续处理。

数据前处理:

使用 `letterbox` 函数对图像进行缩放和填充, 以适应模型的输入尺寸。

初始化识别结果列表:

创建一个空列表 `ill_sets`, 用于存储推理结果。

图像格式转换:

将图像从 BGR 颜色空间转换为 RGB, 并将数据格式从 HWC (高度、宽度、通道) 转换为 CHW (通道、高度、宽度)。

数据归一化:

将图像数据转换为内存连续的数组, 并进行归一化处理。

模型推理:

使用 `model.infer` 方法对图像进行推理, 获取模型的输出。

后处理:

将推理结果从设备内存转移到主机内存, 并转换为 NumPy 数组。

使用非极大值抑制 (NMS) 算法处理模型输出, 以去除重叠的检测框。

坐标缩放:

将 NMS 处理后的检测框坐标缩放回原始图像的大小。

解析识别结果:

遍历处理后的识别结果, 记录序列号、类别 ID 和识别率, 并添加到 `ill_sets` 列表。

返回识别结果：

如果 `ill_sets` 为空，则不执行任何操作；否则，返回包含识别结果的 `ill_sets` 列表。

异常处理：

使用 `try-except` 结构捕获 `KeyboardInterrupt` 异常，允许用户通过 `Ctrl+C` 中断程序。

确保资源释放：

`finally` 块可以放置清理资源的代码。

8.9. 基本信息的初始化

```
1.     if __name__ == "__main__":
2.
3.         # 变量初始化
4.         DEVICE_ID = 0
5.
6.         # 设备id
7.
8.         model_path = 'smart_logistics_models/1.om'
9.
10.        # 模型路径
11.
12.        label_path = 'smart_logistics_models/predefined_classes.txt'
13.
14.        # 标签地址
15.
16.
17.
18.        # 模型与标签txt的初始化
19.
20.        model, labels_dict = model_init(model_path, DEVICE_ID, label_
21.        path)
22.
23.
24.
25.
26.        # 建立通讯
27.
28.
29.        sfd, target_address = network_utils.setup_socket_and_address(
30.
31.        )
32.
33.
34.
35.
36.        # 建立心跳包线程，按照心跳函数中的频率发送心跳包
```

```

15.         heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbea
t.send_udp_heartbeat, 0.25, None, sfd, target_address)
16.         heartbeat_thread.start()
17.
18.         # 站立指令
19.         SendToCommand.perform_action(sfd, target_address, 0x21010202,
0, 0)
20.
21.         # 创建一个简单指令的结构体实例
22.         action_CommandHead = CommandHead()

```

1. 脚本入口点

`if __name__ == "__main__":` 这一行是 Python 中的常规用法, 确保当脚本被直接运行时, 下面的代码块会被执行。

2. 建立通信

`sfd, target_address = network_utils.setup_socket_and_address()`: 这行代码调用了 `network_utils` 模块中的 `setup_socket_and_address` 函数, 用于初始化网络通信所需的套接字 (`sfd`) 和目标地址 (`target_address`)。这用于后续的数据发送和接收。

3. 建立心跳包线程

`heartbeat_thread = MyRepeatThread(...)`: 创建了一个名为 `HeartbeatThread` 的 `MyRepeatThread` 线程实例, 用于定期发送心跳包。心跳包是一种网络通信机制, 用于检测连接是否仍然有效。

`heartbeat.send_udp_heartbeat`: 这是心跳线程将要执行的动作函数, 它使用 UDP 协议发送心跳包。

`0.25`: 心跳包发送的间隔时间, 这里设置为每 0.25 秒发送一次。

`None`: 没有设置最大时间阈值, 因此心跳线程将一直运行直到被外部停止。

`sfd, target_address`: 作为参数传递给心跳函数, 用于指定发送心跳包的套接字和目标地址。

4. 发送站立指令

`SendToCommand.perform_action(sfd, target_address, 0x21010202, 0, 0)`: 这行代码调用了 `SendToCommand` 模块中的 `perform_action` 函数, 发送了一个站立指令到指定的设备。这里

的 0x21010202 是一个指令码，用于告诉设备执行站立动作。

5. 变量初始化

DEVICE_ID: 设备 ID，用于标识特定的设备或机器人。

model_path 和 label_path: 分别指定了模型文件和标签文件的路径。这些文件用于机器学习或模式识别任务。

6. 模型与标签 txt 的初始化

model, labels_dict = model_init(model_path, DEVICE_ID, label_path): 调用 model_init 函数来加载模型和标签字典。

7. 创建指令结构体实例

action_CommandHead = CommandHead(): 创建了一个 CommandHead 类型的实例，这是用于构造或解析发送给设备的命令。

这段代码演示了如何设置网络通信、启动心跳线程、发送特定的控制指令、初始化模型和标签，以及创建用于发送命令的结构体实例。

8.10. 物流卡片判定逻辑

判断是否连续三次检测到相同的手势，并设置标志位 verifications，从而进入机器狗的动作执行循环。

```
1.     verifications = 0  # 进入狗动作的标志位
2.         list0 = []      # 存放手势值的列表
3.         k = 0          # 计数
4.         while True:
5.             ill_sets = Image_inference(model, labels_dict)
6.             print("ill_sets: ", ill_sets)
7.             # print('ill_sets 的长度: ', len(ill_sets))
8.             list0.append(ill_sets[0][1])
9.             k += 1
10.            if k == 3:
11.                # 将列表转换为集合，集合中的元素都是唯一的
12.                unique_elements = set(list0)
```

```

13.
14.             # 如果集合中的元素个数为1，则表示列表中的三个元素都相同
15.             if len(unique_elements) == 1:
16.                 verifications = 1 # 当连续三次都是统一个手势，标志
位归1，进入到动作循环中
17.                 k, list0 = 0, [] # 归0，重新计数
18.             else:
19.                 k, list0 = 0, []
20.             pass

```

代码的位于 `gesture_main.py` 中，解释如下所示：

初始化变量：

verifications：标志位，初始值为 0，用于判断是否连续三次检测到相同的手势。

list0：空列表，用于存储手势值。

k：计数器，初始值为 0，用于记录连续检测到相同手势的次数。

无限循环：

使用 `while True` 创建一个无限循环，这将不断执行循环内的代码。

调用图像推理函数：

调用 `Image_inference(model, labels_dict)` 函数进行图像推理，获取识别结果，并将其存储在 `ill_sets` 变量中。

打印识别结果：

使用 `print` 函数打印 `ill_sets` 的内容。

存储手势值：

如果 `ill_sets` 不为空，将第一个识别结果的类别 ID (`ill_sets[0][1]`) 添加到 `list0` 列表中。

递增计数器：

将计数器 `k` 递增。

判断是否连续三次相同手势：

当计数器 `k` 等于 3 时，执行以下操作：

将 `list0` 转换为集合 `unique_elements`，以去除重复元素并检查元素是否唯一。

如果 `unique_elements` 的长度为 1，说明三次手势相同，将 `verifications` 设置为 1，表示可以执行机械狗的动作。

重置计数器 k 和列表 list0 为初始状态。

重置逻辑：

如果 unique_elements 的长度不为 1，即三次手势不相同，重置 k 和 list0 但不改变 verifications 的值，继续循环以重新计数和检测。

动作执行循环：

当 verifications 为 1 时，根据机械狗识别到的手势，进入动作执行循环中做出相应的动作

上面的循环就是动作执行的循环

8.11. 巡仓与返回逻辑

用于在 verifications 标志位为 ture 时，执行巡仓和返回起点的逻辑。以下是代码的详细解释：

```
1. # 假设存在一个循环，用于执行一系列的验证
2. while verifications:
3.
4.     # 定义目标位置的列表，其中每个位置由角度和距离组成，例如
       [[0, 1.5], [45, 1.0], ...]
5.     # 表示先直走 1.5 米，然后右转 45 度，再走 1.0 米，依此类推
6.     Target_location = [[0, 1.5], [45, 1.0], [0, 0.9], [45, 0.6]]
7.
8.     # 定义不同物品种类对应的目标区域，例如蔬菜、机械、饮品和家具
9.     # 这里的 object_class 变量代表当前物品的种类编号
10.    # actions_params 字典定义了每种物品种类对应的动作参数
11.    actions_params = {
12.        1: [(Target_location[object_class][1],)], # 对于蔬菜，只
           定义了前进的距离
13.        3: [(Target_location[object_class][0],)] # 对于机械，只定义
           了转向的角度
14.    }
```

```

15.
16.     # 定义动作序列, 包括动作 ID 和参数列表索引
17.     # 例如, (3, 0) 表示执行动作 ID 为 3 的动作, 使用索引 0 的参数
18.     actions_sequence = [(3, 0), (1, 0)] # 先转弯, 再前进
19.
20.     # 根据当前位置和目标位置的角度差, 定义返回起点的逻辑
21.     # 这里使用了一系列的条件判断来计算返回路径的角度
22.     if Target_location[object_class][0] == 0:
23.         # 如果当前位置和目标位置角度相同, 直接返回
24.         Target_location_return = [180, Target_location[object_class][1], -180]
25.     elif 0 < Target_location[object_class][0] <= 90:
26.         # 如果角度在 0 到 90 度之间, 计算返回路径的角度
27.         Target_location_return = [180, Target_location[object_class][1], (90+Target_location[object_class][0])]
28.     elif 90 < Target_location[object_class][0] <= 180:
29.         # 如果角度在 90 到 180 度之间, 计算返回路径的角度
30.         Target_location_return = [180, Target_location[object_class][1], (180-Target_location[object_class][0])]
31.     elif -90 < Target_location[object_class][0] < 0:
32.         # 如果角度在 -90 到 0 度之间, 计算返回路径的角度
33.         Target_location_return = [180, Target_location[object_class][1], -(90+abs(Target_location[object_class][0]))]
34.     elif -180 < Target_location[object_class][0] <= -90:
35.         # 如果角度在 -180 到 -90 度之间, 计算返回路径的角度
36.         Target_location_return = [180, Target_location[object_class][1], -(180-abs(Target_location[object_class][0]))]
37.
38.     # 打印返回起点的路径
39.     print(Target_location_return)

```

```

40.
41.     # 定义返回起点的动作参数字典
42.     # 每个动作 ID 对应不同的返回路径参数
43.     actions_params_return = {
44.         3: [(Target_location_return[0], ), (Target_location_return[2], )],
45.         1: [(Target_location_return[1], )],
46.     }
47.
48.     # 定义返回起点的动作序列
49.     actions_sequence_return = [(3, 0), (1, 0), (3, 1)] # 先向右转,
再前进, 最后再向右转

```

目标位置初始化 (Target_location):

这个列表定义了四个目标位置，每个位置由角度和距离组成。例如，`[-45, 1.5]` 表示机器人应该右转 45 度，然后前进 1.5 米。

动作参数字典 (actions_params):

这个字典为不同的物品类别定义了动作参数。

动作序列 (actions_sequence):

这个列表定义了机器人执行动作的顺序。每个元组的第一个元素是动作 ID，第二个元素是 `actions_params` 中对应参数的索引。例如，`(3, 0)` 表示执行动作 ID 为 3 的动作，使用索引 0 的参数。

返回起点的逻辑:

这部分代码根据目标位置的角度来计算返回起点的路径。它使用了多个条件判断来处理不同的角度范围，并计算出返回起点所需的角度和距离。

返回起点的目标位置 (Target_location_return):

根据目标位置的角度，计算出返回起点的角度和距离。这里是具体的计算方法:

如果角度为 0，即机器人直接面对起点，那么它需要先直走，然后右转 180 度回到起点。

如果角度在 0 到 90 度之间，机器人需要先直走，然后右转 90 度加上目标位置的角度，以回到起点。

如果角度在 90 到 180 度之间，机器人需要先直走，然后左转 180 度减去目标位置的角

度。

对于负角度，逻辑类似，但是方向相反。

返回起点的动作参数字典 (`actions_params_return`):

这个字典定义了返回起点时的动作参数。它包含了两个转弯动作和前进动作，每个动作都有对应的参数索引。

返回起点的动作序列 (`actions_sequence_return`):

这个列表定义了返回起点时的动作执行顺序。机器人首先执行第一个转弯，然后前进，最后执行第二个转弯。

打印返回起点的目标位置 (`print(Target_location_return)`):

这行代码用于打印出返回起点时的目标位置参数，这有助于调试和验证计算结果。

8.12. 执行去仓库动作序列

这部分主要用于执行机器人去仓库的动作序列以及执行机器人返回原点的动作序列，详细代码解释如下：

```
1.         for action_id, params_index in actions_sequence:
2.             action_func = actions_dict.get(action_id, None)
3.             if action_func:
4.                 # 从 actions_params 中获取指定动作 ID 和参数索引对
应的参数
5.                 params = actions_params[action_id][params_index]
6.                 logging.info('-----
-----
-----')
7.                 logging.info(f'action_func: {action_func}')
8.                 logging.info(f'params: {params}')
9.                 if isinstance(params, tuple):
10.                     # 调用动作函数，提供已知参数，允许函数使用默认参数值完成调用
```



```

11.                                thread = action_func(*params) # 正确# 展
开参数列表并传递给函数
12.                                thread.start()
13.                                thread.join()
14.                                time.sleep(1)

```

动作序列循环 (for action_id, params_index in actions_sequence):

这个循环遍历由动作 ID 和参数索引组成的列表 actions_sequence。

获取动作函数 (action_func = actions_dict.get(action_id, None)):

使用 actions_dict 字典，根据当前的动作 ID (action_id) 获取对应的动作函数。如果找不到对应的动作 ID，action_func 将被设置为 None。

检查动作函数是否存在 (if action_func):

如果找到了动作函数，代码将继续执行；否则，将跳过当前循环迭代。

获取动作参数 (params = actions_params[action_id][params_index]):

根据当前的动作 ID 和参数索引，从 actions_params 字典中获取相应的参数。

日志记录:

使用 logging.info 记录动作函数和参数的信息，以便于调试和追踪程序执行过程。

参数检查和函数调用 (if isinstance(params, tuple)):

如果参数是一个元组，这表示函数需要多个参数。使用 *params 将元组展开为多个参数，并调用动作函数。

线程创建和启动 (thread = action_func(*params)):

创建一个线程来执行动作函数，并立即启动这个线程。

等待线程完成 (thread.join()):

等待线程执行完成。这确保了在继续下一个动作之前，当前动作已经完全执行。

暂停一秒 (time.sleep(1)):

在执行下一个动作之前，程序暂停一秒，是为了确保动作之间的间隔。

执行返回原点动作序列

```

1.                                for action_id, params_index in actions_sequence_return:
n:

```

```

2.             action_func = actions_dict.get(action_id, None)
3.             if action_func:
4.                 # 从 actions_params 中获取指定动作 ID 和参数索引对
应的参数
5.                 params = actions_params_return[action_id][par
ams_index]
6.                 logging.info('-----
-----')
7.                 logging.info(f'action_func: {action_func}')
8.                 logging.info(f'params: {params}')
9.                 if isinstance(params, tuple):
10.                    # 调用动作函数，提供已知参数，允许函数使用默
认参数值完成调用
11.                    thread = action_func(*params) # 正确# 展
开参数列表并传递给函数
12.                    thread.start()
13.                    thread.join()
14.                    time.sleep(1)
15.
16.
17.                 verifications = 0 # 动作结束，跳出循环，重新开始识别
18.                 break

```

返回起点的动作序列循环 (for action_id, params_index in actions_sequence_return):

这个循环与前面的动作序列循环类似，但是它遍历的是 actions_sequence_return 列表，用于执行返回起点的动作。

重复动作函数调用:

与前面相同，这里再次获取动作函数，获取参数，并在新线程中调用这些函数。

动作结束，重新开始识别 (verifications = 0):

将 verifications 变量设置为 0，这是用来控制循环的退出条件。当 verifications 为非零

值时，循环将继续执行。

跳出循环 (break):

使用 `break` 语句跳出循环，这通常在满足特定条件后使用，以提前结束循环。

第九章 运行示例

首先在文件夹的目录下执行先执行这个指令

```
. /usr/local/Ascend/mxVision-
```

```
5.0.RC3/set_env.sh && . /usr/local/Ascend/ascend-toolkit/set_env.sh
```

然后再输入

```
python smart_logistics_main.py
```

在对机器狗出示卡片即可。

```
(base) root@davinci-mini:/opt/lab/SmartLogistics# python smart_logistics_main.py
Begin to initialize Log.
The output directory of logs file exist.
WARNING: Logging before InitGoogleLogging() is written to STDERR
I20240703 15:26:44.145509 149898 FileUtils.cpp:331] The input file is empty
I20240703 15:26:44.145573 149898 FileUtils.cpp:475] Check Other group permission: Current permission is 4, but required no greater
Save logs information to specified directory.
Find 1 devices!

u3v device: [0]
device model name: MV-CS060-10UC-PRO
user serial number: DA0275184
/usr/local/miniconda3/lib/python3.9/site-packages/torchvision/io/image.py:13: UserWarning: Failed to load image Python extension:
  warn(f"Failed to load image Python extension: {e}")
Starting HeartbeatThread
INFO:root:0 vegetables
ill_sets: [[0, 0, 0.45]]
INFO:root:0 vegetables
ill_sets: [[0, 0, 0.64]]
INFO:root:0 vegetables
ill_sets: [[0, 0, 0.9]]
[180, 1.5, -180]
INFO:root:-----
INFO:root:action_func: <function action_revolve_left_and_right at 0xe7ffcd809d0>
INFO:root:params: (0,)
Starting ACTION_revolve_left_and_right
INFO:root:ACTION_revolve_left_and_right由于超过时间阈值0秒，系统自动停止！
INFO:root:离开线程: ACTION_revolve_left_and_right
INFO:root:-----
INFO:root:action_func: <function action_go_straight at 0xe7ffcd874a60>
INFO:root:params: (1.5,)
Starting ACTION_pan_back_and_forth_thread
INFO:root:ACTION_pan_back_and_forth_thread由于超过时间阈值6.243964167970973秒，系统自动停止！
INFO:root:离开线程: ACTION_pan_back_and_forth_thread
```